CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY

ELECTRONICS & COMMUNICATION ENGINEERING

Major Project Appendix

Title of the Project: Design and RTL-to-GDSII Realization of a High-Performance, Power-Efficient 16-Bit Pipelined RISC Processor

Details of Team Members

S.No	Names	Roll No:	Signatures
1	Nikhil Jindal	2310994537	
2	Sunny Thakral	2310994557	
3	Vedika Kullar	2310994567	
4	Tanmay Sinha	2310994561	

(For official purpose)

Due Date of Report submission	Date of Report submission (To be filled by Major Project In-charge)

Project Coordinator (Sign with Name)

Project Guide (Sign with Name)

Remarks by Guide

CHITKARA UNIVERSITY, PUNJAB

STAGE 1 — INSTRUCTION FETCH (IF)

BLOCK 1 — PROGRAM COUNTER (PC)

What is this Block?

The Program Counter (PC) is a sequential digital circuit responsible for storing the address of the instruction currently being executed by the processor. It acts like a “pointer” that continuously tells the CPU where the next instruction is located inside Instruction Memory.

Without the Program Counter, the processor would not know:

- which instruction to execute,
- where the next instruction is stored,
- or how to move through the program.

The PC is one of the most fundamental blocks in any processor architecture.

Why is this Block Used?

A processor executes instructions sequentially from memory.

The Program Counter is used to:

- fetch instructions in order,
- jump to branch targets,
- pause execution during hazards,
- and stop execution during HALT conditions.

It ensures proper instruction flow across the pipeline.

Role in the Processor

Inside this 5-stage pipelined processor, the Program Counter performs the following functions:

- Supplies instruction address to Instruction Memory.
- Advances execution to next instruction.
- Redirects execution during branches/jumps.
- Holds execution during pipeline stalls.
- Stops execution during HALT instruction.

This block forms the starting point of the entire processor pipeline.

Key Features

1. Sequential Instruction Execution

Automatically increments PC every clock cycle.

2. Branch Handling

Loads branch target address when branch is taken.

3. Stall Support

Freezes PC during hazards or multi-cycle ALU operations.

4. Halt Support
Stops execution completely when HALT instruction reaches WB stage.

5. Synchronous Operation
Updates only on a positive clock edge.

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock signal
reset	1-bit	Resets PC to zero
stall	1-bit	Holds current PC value during hazard/stall
halt	1-bit	Stops processor execution
branch_taken	1-bit	Indicates branch/jump is taken
branch_addr	8-bit	Target branch address

Output Signals

Signal Name	Width	Description
pc	8-bit	Current instruction address

Internal Working Principle
The Program Counter operates sequentially on every positive edge of the clock.

Case 1 — Reset Condition
When **`reset = 1`**:

- PC becomes `0`
- Processor starts execution from address `0`

Case 2 — Stall or Halt
When either: **stall = 1 or halt = 1**
the PC value remains unchanged.
This prevents incorrect instruction fetching during hazards or halted execution.

Case 3 — Branch Taken

When: **branch_taken = 1**
PC loads: branch_addr
This redirects execution flow to branch target instruction.

Case 4 — Normal Execution

If no reset, stall, halt, or branch occurs:
pc <= pc + 1
The processor fetches the next sequential instruction.

Importance in Pipeline

The Program Counter directly affects:

- instruction sequencing,
- branch execution,
- hazard management,
- and pipeline synchronization.

Any incorrect PC update can completely break processor execution flow.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Program Counter (PC)
// ========================================
module program_counter (
    input clk,
    input reset,
    input stall,
    input halt,
    input branch_taken,
    input [7:0] branch_addr,
    output reg [7:0] pc
);

always @(posedge clk or posedge reset) begin
    if (reset)
        pc <= 8'd0;
    else if (halt || stall)
        pc <= pc;
    else if (branch_taken)
        pc <= branch_addr;
    else
        pc <= pc + 1;      // Next instruction
end
endmodule
```

BLOCK 2 — INSTRUCTION MEMORY (IMEM)

What is this Block?

Instruction Memory (IMEM) is a memory block that stores all machine instructions executed by the processor. It acts as the program storage unit of the CPU. Every instruction executed by the processor is fetched from this block using the address generated by the Program Counter (PC). In this processor, the Instruction Memory contains:

- arithmetic instructions,
- logical instructions,
- memory operations,
- stack operations,
- branch instructions,
- and hazard-testing sequences.

Why is this Block Used?

A processor requires a dedicated memory block that stores the program instructions permanently during execution.

Instruction Memory is used because:

- the CPU must fetch instructions continuously,
- instructions must be available quickly,
- and the pipeline requires a new instruction every clock cycle.

Without Instruction Memory, the processor would have no executable program.

Role in the Processor

Inside the processor pipeline, Instruction Memory performs the following tasks:

- Stores the complete instruction sequence.
- Supplies 16-bit instructions to the IF stage.
- Interfaces directly with the Program Counter.
- Provides instruction stream to pipeline.

This block forms the instruction source for the entire CPU.

Key Features

1. 16-bit Instruction Width

Each instruction is 16 bits wide.

2. 8-bit Addressing

Supports 256 instruction locations.

3. Complete ISA Verification Program

Contains:

- ALU tests

- Logic tests
- Stack tests
- Hazard tests
- Branch tests

4. Combinational Read
Instructions are fetched immediately when address changes.

5. Pipeline-Compatible Design
Provides continuous instruction flow for pipelined execution.

Input Signals

Signal Name	Width	Description
addr	8-bit	Instruction address from Program Counter

Output Signals

Signal Name	Width	Description
instr	16-bit	Fetched machine instruction

Internal Working Principle
The Instruction Memory works as a combinational ROM-style block.

Step 1 — Address Input
The Program Counter sends: **addr**
to Instruction Memory.

Step 2 — Instruction Selection
Using: **case(addr)**
the memory selects the corresponding instruction.

Step 3 — Instruction Output
The selected instruction is assigned to: **instr**
which is then forwarded to:
- IF/ID pipeline register
- Instruction Decode stage

Instruction Categories Stored
The memory program contains multiple instruction categories for complete processor verification.

Arithmetic Operations

- ADD
- SUB
- MUL
- DIV
- INC
- DEC
- CMP
- ABS

Logic Operations

- AND
- OR
- XOR
- NOT
- SHL
- SHR
- ROR
- ROL

Memory Operations

- LOAD
- STORE
- ADD
- LOADI
- LOADI
- LOADI
- LOADI

Stack Operations

- PUSH
- POP
- PEAK
- CLEAR

Branch Operations

- JMP
- JZ
- JNZ
- JC
- CMP

Pipeline Importance

Instruction Memory is critical because:

- every pipeline cycle begins with instruction fetch,
- incorrect instruction fetch breaks entire execution,
- and pipeline throughput depends on continuous instruction delivery.

Special Design Feature

This Instruction Memory was intentionally populated with:

- dependency chains,
- hazard conditions,
- stack sequences,
- and branch loops

to fully validate:

- forwarding logic,
- hazard detection,
- branch flushing,
- and multi-cycle stall handling.

This makes the processor verification significantly stronger than simple arithmetic-only testing.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Instruction Memory
// ========================================
module instruction_memory (
    input  [7:0] addr,
    output reg [15:0] instr
);
always @(*) begin
    case(addr)

        // ===== INITIAL VALUES =====
        8'd0:  instr = 16'b10_011_001_000_0_0101; // LOADI R1 = 5
        8'd1:  instr = 16'b10_011_010_000_0_0011; // LOADI R2 = 3
        8'd2:  instr = 16'b10_011_011_000_0_0110; // LOADI R3 = 6
        8'd3:  instr = 16'b10_011_100_000_0_0010; // LOADI R4 = 2

        // ===== ALU DEPENDENCY CHAIN =====
        8'd4:  instr = 16'b00_000_101_001_010; // ADD  R5 = R1 + R2
        8'd5:  instr = 16'b00_001_110_101_011; // SUB  R6 = R5 - R3
        8'd6:  instr = 16'b00_010_111_110_001; // MUL  R7 = R6 * R1
        8'd7:  instr = 16'b00_011_001_111_010; // DIV  R1 = R7 / R2
        8'd8:  instr = 16'b00_100_001_000_000; // INC
```

```
        8'd9:  instr = 16'b00_101_010_000_000; // DEC
        8'd10: instr = 16'b00_110_001_010_000; // CMP
        8'd11: instr = 16'b00_111_011_000_000; // ABS

        // ===== LOGIC =====
        8'd12: instr = 16'b01_000_001_010_011; // AND
        8'd13: instr = 16'b01_001_010_011_100; // OR
        8'd14: instr = 16'b01_010_001_000_000; // NOT
        8'd15: instr = 16'b01_011_001_010_011; // XOR
        8'd16: instr = 16'b01_100_001_000_000; // SHL
        8'd17: instr = 16'b01_101_001_000_000; // SHR
        8'd18: instr = 16'b01_110_001_000_000; // ROR
        8'd19: instr = 16'b01_111_001_000_000; // ROL

        // ===== MEMORY + HAZARD =====
        8'd20: instr = 16'b10_001_111_001_010; // STORE R7 → [R1]
        8'd21: instr = 16'b10_000_011_001_010; // LOAD  R3 ← [R1]
        8'd22: instr = 16'b00_000_100_011_001; // ADD (load-use hazard)

        // ===== STACK =====
        8'd23: instr = 16'b10_100_100_000_000; // PUSH
        8'd24: instr = 16'b10_101_110_000_000; // POP
        8'd25: instr = 16'b10_110_001_000_000; // PEAK
        8'd26: instr = 16'b10_111_001_000_000; // CLEAR

        // ===== BRANCH (ALL PATHS) =====
        8'd27: instr = 16'b00_110_110_001_010; // CMP R6,R2
        8'd28: instr = 16'b11_001_000_000_0_0011; // JZ
        8'd29: instr = 16'b11_010_000_000_0_0010; // JNZ
        8'd30: instr = 16'b11_011_000_000_0_0010; // JC

        // ===== LOOP =====
        8'd31: instr = 16'b11_000_000_000_0_1110; // JMP backward

        default: instr = 16'b11_111_000_000_0_0000;

    endcase
end
endmodule
```

BLOCK 3 — IF/ID PIPELINE REGISTER

What is this Block?

The IF/ID Pipeline Register is a sequential storage block placed between the Instruction Fetch (IF) stage and the Instruction Decode (ID) stage.
It temporarily stores:

- fetched instruction,
- and corresponding Program Counter value

before passing them to the next pipeline stage.
This block acts as a synchronization boundary between IF and ID stages.

Why is this Block Used?

In a pipelined processor, multiple instructions execute simultaneously in different stages.
Without pipeline registers:

- signals would change immediately,
- stages would overlap incorrectly,
- and pipeline timing would fail.

The IF/ID register is used to:

- isolate IF stage from ID stage,
- maintain proper clock synchronization,
- and allow multiple instructions to move independently through the pipeline.

Role in the Processor

The IF/ID register performs the following functions:

- Stores fetched instructions from Instruction Memory.
- Stores current Program Counter value.
- Transfers stable data to Decode stage.
- Supports pipeline stalls.
- Supports pipeline flushing during branch operations.

This block is essential for maintaining correct pipelined execution.

Key Features

1. Pipeline Synchronization

Separates IF and ID stages using clocked storage.

2. Stall Support

Holds current values during hazards or ALU stalls.

3. Flush Support

Inserts NOP instruction during branch redirection.

4. Stable Data Transfer
Ensures Decode stage receives stable instruction data.

5. Sequential Operation
Updates only on a positive clock edge.

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock
reset	1-bit	Clears register contents
stall	1-bit	Holds current pipeline values
flush	1-bit	Inserts NOP instruction
pc_in	8-bit	PC value from IF stage
instr_in	16-bit	Instruction from Instruction Memory

Output Signals

Signal Name	Width	Description
pc_out	8-bit	Stored PC value for ID stage
instr_out	16-bit	Stored instruction for ID stage

Internal Working Principle
The IF/ID register captures instruction and PC information on every positive clock edge.

Case 1 — Reset Condition
When: **reset = 1**
all outputs become zero. This initializes the pipeline safely.

Case 2 — Stall Condition
When: **stall = 1**
the register holds its previous values. This prevents incorrect instruction movement during:
- hazards,
- forwarding delays,
- or multi-cycle operations.

Case 3 — Flush Condition

When: **flush = 1**
the register inserts a NOP instruction: **16'b11_011_000_000_0_0000**
This removes incorrectly fetched instructions after branch decisions.

Case 4 — Normal Pipeline Operation

During normal execution:

- PC and instruction values are latched,
- then forwarded to the Decode stage.

Why Flush is Important

Branches are resolved later in the pipeline.
During that delay:

- wrong instructions may already be fetched.

Flush logic removes these incorrect instructions to maintain proper execution flow.

Without flushing:

- incorrect instructions could execute,
- causing invalid processor behavior.

Importance in Pipeline Architecture

The IF/ID register is critical because it:

- enables pipelined execution,
- isolates timing between stages,
- supports hazard handling,
- and enables branch correction.

Without this block, proper 5-stage pipelining would not be possible.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : IF/ID Pipeline Register
// ========================================
module if_id_reg (
    input clk,
    input reset,
    input stall,
    input flush,
    input [7:0] pc_in,
    input [15:0] instr_in,
    output reg [7:0] pc_out,
    output reg [15:0] instr_out
);
```

```
always @(posedge clk or posedge reset) begin
    if (reset) begin
        pc_out <= 0;
        instr_out <= 0;
    end
    else if (stall) begin
        pc_out <= pc_out;       // Hold value
        instr_out <= instr_out; // Hold value
    end
    else if (flush) begin
        instr_out <= 16'b11_011_000_000_0_0000; // NOP
        pc_out <= pc_out;  // keep PC instead of 0
    end
    else begin
        pc_out <= pc_in;
        instr_out <= instr_in;
    end
end

endmodule
```

BLOCK 4 — FLUSH UNIT

What is this Block?

The Flush Unit is a combinational control logic block responsible for removing incorrect instructions from the pipeline after a branch or jump operation occurs. In a pipelined processor, instructions are fetched continuously before the processor fully knows whether a branch will be taken or not. This can cause incorrect instructions to enter the pipeline. The Flush Unit solves this problem.

Why is this Block Used?

Branch instructions create a problem called a:
Control Hazard
This happens because:

- the processor fetches instructions sequentially,
- but later discovers that execution must jump somewhere else.

As a result:

- some already-fetched instructions become invalid.

The Flush Unit is used to:

- remove those incorrect instructions,
- prevent wrong execution,
- and maintain proper program flow.

Without flushing:

- incorrect instructions could execute,
- registers could get corrupted,
- and pipeline behavior would become invalid.

Role in the Processor

The Flush Unit performs the following functions:

- Detects branch redirection events.
- Generates flush signals.
- Removes wrong-path instructions.
- Maintains correct control flow.

This block works closely with:

- Branch Unit,
- IF/ID pipeline register,
- and pipeline control logic.

Key Features

1. Simple Combinational Design

No clock required.

2. Branch-Controlled Flushing

Activated only when a branch is taken.

3. Pipeline Recovery

Prevents execution of invalid instructions.

4. Fast Operation

Flush signal generated immediately.

5. Minimal Hardware Complexity

Very lightweight implementation.

Input Signals

Signal Name	Width	Description
branch_taken	1-bit	Indicates branch or jump taken

Output Signals

Signal Name	Width	Description
flush	1-bit	Flush signal for pipeline register

Internal Working Principle

The Flush Unit directly maps:
branch_taken → flush

Case 1 — No Branch Taken

When: **branch_taken = 0**
Then: **flush = 0**
The pipeline continues normally.

Case 2 — Branch Taken

When: **branch_taken = 1**
Then: **flush = 1**
The IF/ID pipeline register inserts a NOP instruction.
This removes incorrectly fetched instructions from the pipeline.

Pipeline Importance

The Flush Unit is essential because:

- branches are resolved in later stages,
- but instruction fetching happens earlier.

This timing mismatch creates control hazards.

The Flush Unit ensures:

- only valid instructions continue execution,
- and pipeline execution remains correct.

Special Design Advantage

This processor uses:
flush = branch_taken
which makes the design:

- simple,
- fast,
- synthesizable,
- and easy to debug.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Flush Unit
// ========================================
module flush_unit (
    input branch_taken,
    output flush
);

assign flush = branch_taken;
endmodule
```

BLOCK 5 — STACK POINTER (SP)

What is this Block?

The Stack Pointer (SP) is a special-purpose register used to manage stack operations inside the processor.
It stores the address of the current top location of the stack memory.
The stack is a dedicated memory structure used for:

- temporary data storage,
- function-like operations,
- PUSH/POP operations,
- and intermediate data management.

The Stack Pointer always points to the current active stack location.

Why is this Block Used?

Certain processor operations require temporary storage that follows:
Last-In First-Out (LIFO)
behavior.
This is called a stack.
The Stack Pointer is used because:

- the processor must know where stack data is stored,
- stack operations continuously change stack position,
- and automatic stack management is required for PUSH and POP instructions.

Without the Stack Pointer:

- stack operations would become impossible,
- memory locations could overlap,
- and data corruption would occur.

Role in the Processor

Inside this processor, the Stack Pointer performs the following functions:

- Tracks current stack top address.
- Updates stack location during PUSH operations.
- Updates stack location during POP operations.
- Interfaces with Data Memory address generation.
- Supports stack-based instructions.

This block directly supports:

- PUSH
- POP
- PEAK
- CLEAR

instructions.

Key Features

1. Dedicated Stack Address Tracking

Maintains current stack location.

2. PUSH Support

Moves stack downward during data push.

3. POP Support

Moves stack upward during data pop.

4. Overflow and Underflow Protection

Prevents invalid stack movement.

5. Sequential Operation

Updates synchronously with clock.

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock
reset	1-bit	Resets Stack Pointer
push	1-bit	Indicates PUSH operation
pop	1-bit	Indicates POP operation

Output Signals

Signal Name	Width	Description
sp	8-bit	Current stack pointer value

Internal Working Principle

The Stack Pointer updates on every positive clock edge.

Case 1 — Reset Condition

When: **reset = 1**
the Stack Pointer becomes: **8'hFF**
This initializes the stack at the top of memory.

Case 2 — PUSH Operation

When: **push = 1**
the stack pointer decreases: **sp <= sp - 1**
This is because the stack grows downward in memory.

Case 3 — POP Operation

When: **pop = 1**
the stack pointer increases: **sp <= sp + 1**
This removes the top stack entry.

Case 4 — No Stack Operation

If neither PUSH nor POP occurs: **sp <= sp**
The current stack position remains unchanged.

Overflow and Underflow Protection

The design includes safety protection.

PUSH Protection

Stack pointer does not decrement below: **0**

POP Protection

Stack pointer does not increment above: **8'hFF**
This prevents invalid memory access.

Stack Growth Direction

This processor uses:
Descending Stack
Meaning:

- PUSH → stack moves downward
- POP → stack moves upward

This is a common processor stack implementation method.

Pipeline Importance

The Stack Pointer is important because:

- stack instructions require dynamic memory addressing,
- stack address changes every operation,
- and correct stack tracking is essential for reliable memory behavior.

Incorrect SP handling can corrupt:

- stack data,
- memory contents,
- and processor execution flow.

Special Design Advantage

This implementation:

- uses minimal hardware,
- supports automatic stack management,
- integrates directly with MEM stage,
- and cleanly interfaces with Data Memory.

The design is fully synthesizable and pipeline compatible.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Stack Pointer (SP)
// ========================================
module stack_pointer (
    input clk,
    input reset,
    input push,
    input pop,
    output reg [7:0] sp
);

always @(posedge clk or posedge reset) begin
    if (reset)
        sp <= 8'hFF;
    else begin
        case ({push, pop})
            2'b10: if (sp != 0) sp <= sp - 1;
            2'b01: if (sp != 8'hFF) sp <= sp + 1;
            default: sp <= sp;
        endcase
    end
end
endmodule
```

STAGE 2 — INSTRUCTION DECODE (ID)

BLOCK 6 — INSTRUCTION DECODER

What is this Block?

The Instruction Decoder is a combinational logic block responsible for separating and interpreting different fields of the 16-bit machine instruction.
It extracts:

- instruction group,
- opcode,
- source registers,
- destination register,
- and immediate values

from the fetched instruction.
This is one of the most important blocks in the processor because it allows the CPU to understand what operation must be performed.

Why is this Block Used?

The processor receives instructions in binary form.
Example: **10_011_001_000_0_0101**
The CPU itself cannot directly understand this binary pattern.

The Instruction Decoder converts this binary instruction into meaningful fields such as:

- operation type,
- register addresses,
- and immediate operands.

Without the decoder:

- the processor would not know which operation to execute,
- which registers to use,
- or which data values are required.

Role in the Processor

The Instruction Decoder performs the following tasks:

- Separates instruction fields.
- Identifies instruction category.
- Extracts ALU operation code.
- Extracts source and destination registers.
- Extracts immediate value.
- Supplies decoded signals to Control Unit and datapath.

This block forms the bridge between:
Instruction Fetch → Instruction Decode

Key Features

1. Fast Combinational Operation

No clock required.

2. ISA-Compatible Decoding

Matches custom 16-bit ISA format.

3. Multi-Group Instruction Support

Supports:

- ALU instructions
- Logic instructions
- Memory instructions
- Branch instructions

4. Register Address Extraction

Directly extracts:

- rs1 and rs2
- rd

5. Immediate Extraction

Provides immediate operand field for immediate-type instructions.

Input Signals

Signal Name	Width	Description
instr	16-bit	Input instruction from IF/ID register

Output Signals

Signal Name	Width	Description
group	2-bit	Instruction category
opcode	3-bit	Operation inside instruction group
rd	3-bit	Destination register address
rs1	3-bit	Source register 1 address
rs2	3-bit	Source register 2 address
imm	5-bit	Immediate value

Instruction Format Used in Processor

The processor uses a custom 16-bit ISA format.

Bit Range	Field	Purpose
[15:14]	group	Instruction category
[13:11]	opcode	Specific operation
[10:8]	rd	Destination register
[7:5]	rs1	Source register 1
[4:2]	rs2	Source register 2
[4:0]	imm	Immediate value

Instruction Groups

Group	Meaning
`00`	ALU Instructions
`01`	Logic Instructions
`10`	Memory and Stack Instructions
`11`	Branch and Control Instructions

Internal Working Principle

The Instruction Decoder works using direct bit extraction.

Group Extraction

assign group = **instr[15:14];**
Determines major instruction category.

Opcode Extraction

assign opcode = **instr[13:11];**
Determines exact operation inside the instruction group.

Register Address Extraction

assign rd = **instr[10:8];**
assign rs1 = **instr[7:5];**
assign rs2 = **instr[4:2];**
Provides register addresses to Register File and datapath.

Immediate Extraction

assign imm = **instr[4:0];**
Used by:

- LOADI
- branch instructions
- immediate operations

Importance in Processor Pipeline

The Instruction Decoder is critical because:

- every instruction passes through this block,
- all control decisions depend on decoded fields,
- and datapath operation completely relies on correct decoding.

Any decoding error can cause:

- incorrect ALU operation,
- wrong memory access,
- or invalid branching behavior.

Special Design Advantage

This decoder design is:

- lightweight,
- fully combinational,
- synthesizable,
- and extremely fast.

It avoids:

- microcode,
- complex FSM decoding,
- and multi-stage instruction parsing.

This improves:

- timing performance,
- area efficiency,
- and pipeline simplicity.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Instruction Decoder
// ========================================
module instruction_decoder (
    input  [15:0] instr,
    output [1:0]  group,
    output [2:0]  opcode,
    output [2:0]  rd,
    output [2:0]  rs1,
    output [2:0]  rs2,
    output [4:0]  imm
);

assign group  = instr[15:14];
assign opcode = instr[13:11];
assign rd     = instr[10:8];
assign rs1    = instr[7:5];
assign rs2    = instr[4:2];
assign imm    = instr[4:0];

endmodule
```

BLOCK 7 — REGISTER FILE (RF)

What is this Block?

The Register File (RF) is a small high-speed storage unit inside the processor that stores temporary data required during instruction execution. It contains the processor's general-purpose registers used by the ALU and datapath.
In this processor:
8 General Purpose Registers
are implemented.

These registers temporarily store:

- operands,
- intermediate results,
- memory data,
- and final execution outputs.

Why is this Block Used?

Accessing the main memory for every operation is very slow.
Registers are used because they provide:

- extremely fast data access,
- low latency,
- and efficient operand storage.

The Register File allows the processor to:

- quickly fetch operands,
- perform computations,
- and store results efficiently.

Without registers:

- processor performance would collapse,
- ALU execution would slow dramatically,
- and pipelining would become inefficient.

Role in the Processor

The Register File performs the following functions:

- Stores processor working data.
- Supplies source operands to ALU.
- Receives final writeback results.
- Provides simultaneous dual-read access.
- Supports pipelined execution.

This block is the primary operand storage unit of the CPU.

Key Features

1. 8 General-Purpose Registers

Registers: **R0 → R7**

2. Dual Read Ports

Can read two operands simultaneously.

3. Single Write Port

Supports one register write per cycle.

4. Synchronous Write Operation

Writes occur only on clock edges.

5. Reset Support

All registers initialize to zero during reset.

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock
reset	1-bit	Clears all registers
reg_write	1-bit	Enables register write
rs1	3-bit	Source register 1 address
rs2	3-bit	Source register 2 address
rd	3-bit	Destination register address
write_data	16-bit	Data to be written into register

Output Signals

Signal Name	Width	Description
read_data1	16-bit	Data from rs1 register
read_data2	16-bit	Data from rs2 register

Internal Working Principle

The Register File supports:

- combinational reading,
- and synchronous writing.

Read Operation

Reads occur continuously using:
assign read_data1 = registers[rs1];
assign read_data2 = registers[rs2];
This allows:

- both operands to become available immediately,
- enabling faster ALU execution.

Write Operation

Writing occurs only on:
positive clock edge
When:
reg_write = 1
the following operation occurs:
registers[rd] <= write_data;
This updates the destination register with the final execution result.

Reset Operation

During reset:

- all registers become zero.

This ensures:

- predictable startup behavior,
- stable pipeline initialization,
- and safe execution.

Register Usage in Processor

The Register File is heavily used throughout the pipeline.

Source Operands

Registers provide:
rs1_data
rs2_data
to:

- Forwarding Unit
- Operand MUX
- ALU

Destination Writeback

Final result from WB stage is written back into: rd register.

Pipeline Importance

The Register File is critical because:

- nearly every instruction interacts with registers,
- ALU operations depend on register operands,
- and writeback stage updates processor state through this block.

Incorrect Register File behavior can corrupt:

- computations,
- memory addresses,
- and control flow.

Special Design Advantage

This Register File implementation:

- supports pipelined operation,
- allows simultaneous operand fetch,
- integrates cleanly with forwarding logic,
- and remains synthesizable and area-efficient.

The dual-read single-write architecture is widely used in real processor designs.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Register File (RF)
// ========================================
module register_file (
    input clk,
    input reset,
    input reg_write,
    input [2:0] rs1, rs2, rd,
    input [15:0] write_data,
    output [15:0] read_data1, read_data2
);

reg [15:0] registers [0:7];
integer i;

// READ
assign read_data1 = registers[rs1];
assign read_data2 = registers[rs2];

// WRITE + RESET
always @(posedge clk or posedge reset) begin
    if (reset) begin
        for (i = 0; i < 8; i = i + 1)
            registers[i] <= 16'd0;
    end
    else if (reg_write) begin
        registers[rd] <= write_data;
    end
end

endmodule
```

BLOCK 8 — IMMEDIATE GENERATOR

What is this Block?

The Immediate Generator is a combinational logic block responsible for converting a small immediate value from the instruction into a full 16-bit operand. In this processor, instructions may contain:
5-bit immediate values

The Immediate Generator expands this smaller value into:
16-bit signed data
using sign extension.

Why is this Block Used?

Some instructions do not require two register operands.
Instead, they directly contain a constant value inside the instruction itself.
Example: **LOADI R1, #5**
Here: **5**
is an immediate value.

The ALU and datapath operate on: **16-bit data**
therefore the immediate value must also become 16 bits before execution.

Without the Immediate Generator:

- immediate instructions would fail,
- ALU operand widths would mismatch,
- and datapath compatibility would break.

Role in the Processor

The Immediate Generator performs the following functions:

- Extracts immediate operand.
- Converts 5-bit immediate into 16-bit value.
- Performs sign extension.
- Supplies immediate operand to Execute stage.

This block is mainly used for:

- LOADI
- branch offsets
- immediate-based instructions

Key Features

1. Sign Extension Support

Preserves positive and negative immediate values.

2. Fully Combinational
No clock required.

3. Lightweight Hardware
Very small area overhead.

4. Pipeline-Compatible
Provides immediate operand directly to ID/EX register.

5. 16-bit Datapath Compatibility
Matches ALU operand width.

Input Signals

Signal Name	Width	Description
imm_in	5-bit	Immediate value extracted from instruction

Output Signals

Signal Name	Width	Description
imm_out	16-bit	Sign-extended immediate value

Internal Working Principle
The Immediate Generator uses:
Sign Extension
to expand the immediate value.

Sign Extension Concept
If the MSB of the immediate is: 1
the value is treated as negative.
The generator copies the sign bit into upper bits.

Implementation Used
The RTL performs: **assign imm_out = {{11{imm_in[4]}}, imm_in};**
This means:
- the sign bit `imm_in[4]`
- is replicated 11 times,
- then concatenated with the original 5-bit immediate.

Example 1 — Positive Immediate

Input: **00101**
Output: **0000000000000101**
Decimal: **+5**

Example 2 — Negative Immediate

Input: **11101**
Output: **1111111111111101**
Decimal: **-3**

Processor Usage

The Immediate Generator output is used by:

- Operand MUX
- ALU
- Branch Unit

It allows instructions to directly operate on constants without requiring extra memory access.

Importance in Processor

This block is critical because:

- immediate instructions are extremely common,
- branch offsets depend on immediate values,
- and ALU operand widths must remain consistent.

Without proper sign extension:

- negative numbers would become incorrect,
- branches could jump incorrectly,
- and ALU computations would fail.

Special Design Advantage

This implementation:

- is extremely area-efficient,
- synthesizes into very small hardware,
- introduces almost zero delay,
- and integrates cleanly with the pipeline.

The design is fully synthesizable and timing-friendly.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Immediate Generator
// ========================================
module immediate_generator (
    input [4:0] imm_in,
    output [15:0] imm_out
);

assign imm_out = {{11{imm_in[4]}}, imm_in};

endmodule
```

BLOCK 9 — CONTROL UNIT

What is this Block?

The Control Unit is the main decision-making logic block of the processor. It acts like the “brain” of the CPU by generating all control signals required for correct instruction execution. After the Instruction Decoder extracts instruction fields, the Control Unit determines:

- what operation must be performed,
- which datapath blocks must activate,
- and how data should move through the processor.

Without the Control Unit, the datapath hardware would exist physically but would not know how to operate.

Why is this Block Used?

Different instructions require different hardware behavior.
For example:

ADD instruction

Needs:

- ALU operation
- register writeback

LOAD instruction

Needs:

- memory read
- register writeback
- memory-to-register selection

STORE instruction

Needs:

- memory write only

The Control Unit generates the correct control signals for every instruction type.

Without it:

- ALU would perform wrong operations,
- memory accesses would fail,
- registers could update incorrectly,
- and processor execution would become invalid.

Role in the Processor

The Control Unit performs the following tasks:

- Decodes instruction behavior.
- Generates ALU control signals.
- Controls memory access.
- Controls register writing.

- Controls stack operations.
- Controls jumps and branching.
- Controls flag updating.

This block controls almost every major datapath component in the processor.

Key Features

1. Multi-Group ISA Support

Supports:

- ALU instructions
- Logic instructions
- Memory instructions
- Stack instructions
- Branch instructions

2. Fully Combinational Design

No clock required.

3. Centralized Control Logic

A single unit controls the entire datapath behavior.

4. Stack Instruction Support

Supports:

- PUSH
- POP
- PEAK
- CLEAR

5. Branch and HALT Support

Handles:

- JMP
- HALT
- conditional branch preparation

Input Signals

Signal Name	Width	Description
group	2-bit	Instruction category
opcode	3-bit	Operation inside group

Output Signals

Signal Name	Width	Description
is_push	1-bit	Indicates PUSH operation
is_pop	1-bit	Indicates POP operation
is_peak	1-bit	Indicates PEAK operation
alu_src	1-bit	Selects ALU operand source
reg_write	1-bit	Enables register write
mem_read	1-bit	Enables memory read
mem_write	1-bit	Enables memory write
mem_to_reg	1-bit	Selects memory data for WB
jump	1-bit	Indicates jump instruction
halt	1-bit	Stops processor execution
alu_op	3-bit	ALU operation select
flag_write	1-bit	Enables flag register update

Internal Working Principle

The Control Unit uses:
group + opcode
to determine instruction behavior.

Instruction Group Decoding

Group 00 — ALU Instructions

Operations:

- ADD
- SUB
- MUL
- DIV
- INC
- DEC
- CMP
- ABS

Control Actions

- Enables ALU execution.
- Enables flag update.
- Enables register writing.

Special Case:
CMP
updates flags but does not write register data.

Group 01 — Logic Instructions

Operations:

- AND
- OR
- XOR
- NOT
- SHL
- SHR
- ROR
- ROL

Control Actions

- Enables ALU logic mode.
- Enables flag update.
- Enables register writing.

Group 10 — Memory and Stack Instructions

Operations:

- LOAD
- STORE
- MOVE
- LOADI
- PUSH
- POP
- PEAK
- CLEAR

Control Actions

Controls:

- memory access,
- stack behavior,
- immediate operations,
- register updates.

Group 11 — Control Instructions

Operations:

- JMP
- HALT
- conditional branches

Control Actions

- Enables branch execution.
- Enables processor halt.

Important Control Signals

alu_src

Determines whether ALU operand comes from:

- register
  or
- immediate value.

mem_to_reg

Determines whether writeback data comes from:

- ALU result
  or
- memory output.

flag_write

Controls whether NZCV flags should update.

jump

Activates branch/jump control flow.

halt

Stops processor execution completely.

Pipeline Importance

The Control Unit is one of the most critical blocks because:

- every datapath action depends on its outputs,
- all stages rely on proper control signaling,
- and incorrect control signals can completely break processor execution.

This block coordinates the behavior of:

- ALU,
- Register File,

- Data Memory,
- Branch Unit,
- Stack Logic,
- and the Writeback stage.

Special Design Advantage

This Control Unit design:

- uses simple combinational logic,
- avoids microcoded control,
- reduces hardware complexity,
- improves synthesis efficiency,
- and provides fast control signal generation.

The architecture is highly suitable for pipelined processors.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Control Unit
// ========================================
module control_unit (
    input [1:0] group,
    input [2:0] opcode,

    output reg is_push,
    output reg is_pop,
    output reg is_peak,

    output reg alu_src,
    output reg reg_write,
    output reg mem_read,
    output reg mem_write,
    output reg mem_to_reg,
    output reg jump,
    output reg halt,
    output reg [2:0] alu_op,
    output reg flag_write
);

always @(*) begin
    // ================= DEFAULT ================
    alu_src     = 0;
    reg_write   = 0;
    mem_read    = 0;
    mem_write   = 0;
    mem_to_reg  = 0;
    jump        = 0;
    halt        = 0;
    alu_op      = 3'b000;
```

```
flag_write  = 0;

is_push     = 0;
is_pop      = 0;
is_peak     = 0;

// ================= DECODE =================
case (group)

   // -------- ALU --------
   2'b00: begin
      reg_write  = 1;
      alu_op     = opcode;
      flag_write = 1;

      if (opcode == 3'b110)   // CMP
         reg_write = 0;
   end

   // -------- LOGIC --------
   2'b01: begin
      reg_write  = 1;
      alu_op     = opcode;
      flag_write = 1;
   end

   // -------- MEMORY + STACK --------
   2'b10: begin
      case (opcode)

         3'b000: begin // LOAD
            mem_read   = 1;
            reg_write  = 1;
            mem_to_reg = 1;
         end

         3'b001: begin // STORE
            mem_write = 1;
         end

         3'b010: begin // MOVE
            reg_write = 1;
         end

         3'b011: begin // LOADI
            alu_src   = 1;
            reg_write = 1;
         end

         3'b100: begin // PUSH
            mem_write = 1;
```

```
            is_push   = 1;
        end

        3'b101: begin // POP
            mem_read   = 1;
            reg_write  = 1;
            mem_to_reg = 1;
            is_pop     = 1;
        end

        3'b110: begin // PEAK
            mem_read   = 1;
            reg_write  = 1;
            mem_to_reg = 1;
            is_peak    = 1;
        end

        3'b111: begin // CLEAR
            reg_write = 1;
        end
      endcase
    end

    // -------- CONTROL --------
    2'b11: begin
      case (opcode)
        3'b000: jump = 1;   // JMP
        3'b111: halt = 1;   // HALT
        default: ;          // conditional branches handled in branch_unit
      endcase
    end

  endcase
end

endmodule
```

BLOCK 10 — HAZARD DETECTION UNIT

What is this Block?

The Hazard Detection Unit is a combinational logic block responsible for detecting situations where pipeline execution may produce incorrect results due to data dependency between instructions.
In a pipelined processor, multiple instructions execute simultaneously.
Sometimes:

- one instruction needs data,
- but that data has not yet been produced by a previous instruction.

This situation is called a: **Data Hazard**
The Hazard Detection Unit detects these conditions and temporarily stalls the pipeline to prevent incorrect execution.

Why is this Block Used?

Pipelining improves performance by executing multiple instructions at the same time.
However, this creates dependency problems.
Example:
LOAD R3, [R1]
ADD R4, R3, R2

The ADD instruction needs: R3
but the LOAD instruction has not yet finished loading data from memory.

If execution continues normally:

- ADD will use invalid data,
- incorrect computation will occur,
- and processor behavior becomes wrong.

The Hazard Detection Unit solves this by:

- detecting the dependency,
- stalling the pipeline,
- and allowing correct data to become available.

Role in the Processor

The Hazard Detection Unit performs the following tasks:

- Detects load-use hazards.
- Monitors register dependencies.
- Generates stall signals.
- Prevents incorrect instruction execution.

This block maintains pipeline correctness during dependent instruction execution.

Key Features

1. RAW Hazard Detection

Detects:
Read After Write (RAW)
Dependencies.

2. Automatic Pipeline Stall

Generates stall without software intervention.

3. Lightweight Combinational Logic

No clock required.

4. Pipeline Safety

Prevents invalid operand usage.

5. Load Hazard Protection

Specially designed for memory read dependencies.

Input Signals

Signal Name	Width	Description
rs1_id	3-bit	Source register 1 in ID stage
rs2_id	3-bit	Source register 2 in ID stage
rd_ex	3-bit	Destination register in EX stage
mem_read_ex	1-bit	Indicates EX stage is performing LOAD

Output Signals

Signal Name	Width	Description
hazard_stall	1-bit	Pipeline stall signal

Internal Working Principle

The Hazard Detection Unit continuously checks whether:

- the current instruction depends on data,
- that is still being produced by a previous LOAD instruction.

Hazard Condition

Hazard occurs when: **mem_read_ex = 1 AND (rd_ex == rs1_id) OR (rd_ex == rs2_id)**
This means:

- current instruction needs data,
- but the previous LOAD instruction has not completed.

Hazard Logic Used

The RTL logic behaves: **hazard_stall = mem_read_ex && ((rd_ex == rs1_id) || (rd_ex == rs2_id));**

Example Hazard Scenario

Instruction 1

LOAD R3, [R1]

Instruction 2

ADD R4, R3, R2
Problem:

- ADD needs R3
- but LOAD has not yet completed the memory read.

Hazard Unit detects this dependency and **hazard_stall = 1**
The pipeline temporarily pauses.

What Happens During a Stall?

When stall is generated:

- Program Counter freezes.
- IF/ID register holds values.
- The pipeline waits one cycle.

After data becomes available:

- execution resumes normally.

Pipeline Importance

The Hazard Detection Unit is critical because:

- pipelined instructions overlap in execution,
- dependencies are extremely common,
- and incorrect hazard handling causes invalid processor results.

Without this block:

- dependent instructions would execute with wrong operands,
- arithmetic results would corrupt,
- and processor reliability would fail.

Relationship with Forwarding Unit

The processor uses:

- Forwarding Unit
  AND
- Hazard Detection Unit

together.

Forwarding Unit

Handles: most ALU dependencies

Hazard Unit

Handles: LOAD-use hazards
because memory data is not immediately available for forwarding.

Special Design Advantage

This implementation:

- is simple,
- fast,
- synthesizable,
- and introduces minimal hardware overhead.

It provides efficient hazard protection while maintaining high pipeline throughput.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Hazard Detection Unit
// ========================================
module hazard_unit (
    input [2:0] rs1_id,
    input [2:0] rs2_id,
    input [2:0] rd_ex,
    input mem_read_ex,
    output reg hazard_stall
);

always @(*) begin
    hazard_stall = 0;

    // Only LOAD-USE hazard
    if (mem_read_ex && (rd_ex != 0) &&
    ((rd_ex == rs1_id) || (rd_ex == rs2_id)))
            hazard_stall = 1;
end
endmodule
```

BLOCK 11 — ID/EX PIPELINE REGISTER

What is this Block?

The ID/EX Pipeline Register is a sequential storage block placed between the Instruction Decode (ID) stage and the Execute (EX) stage of the processor pipeline.
It temporarily stores:

- decoded operands,
- control signals,
- register addresses,
- immediate values,
- and instruction-related data

before forwarding them to the Execute stage.
This block acts as the synchronization boundary between:
Instruction Decode → Execute
stages.

Why is this Block Used?

In a pipelined processor:

- multiple instructions execute simultaneously,
- each stage works independently,
- and data must remain stable between stages.

Without pipeline registers:

- signals would change unpredictably,
- ALU inputs would become unstable,
- and pipeline timing would fail.

The ID/EX register is used to:

- isolate Decode and Execute stages,
- synchronize datapath signals,
- and safely transfer instruction information into the ALU stage.

Role in the Processor

The ID/EX Pipeline Register performs the following tasks:

- Stores decoded register operands.
- Stores immediate values.
- Stores destination register address.
- Stores ALU control signals.
- Stores memory and writeback control signals.
- Transfers stable data to Execute stage.

This block is one of the most important synchronization elements in the entire pipeline.

Key Features

1. Pipeline Synchronization

Separates ID and EX stages.

2. Stable Operand Transfer

Ensures ALU receives stable operands.

3. Control Signal Propagation

Transfers execution control safely.

4. Hazard-Compatible Design

Supports stall handling.

5. Sequential Clocked Storage

Updates only on a positive clock edge.

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock
reset	1-bit	Clears pipeline register
stall	1-bit	Holds current values
pc_in	8-bit	PC from ID stage
rs1_data_in	16-bit	Source operand 1
rs2_data_in	16-bit	Source operand 2
imm_in	16-bit	Immediate operand
rd_in	3-bit	Destination register
rs1_in	3-bit	Source register 1
rs2_in	3-bit	Source register 2
alu_op_in	3-bit	ALU operation control
alu_src_in	1-bit	ALU operand source select
reg_write_in	1-bit	Register write enable

mem_read_in	1-bit	Memory read enable
mem_write_in	1-bit	Memory write enable
mem_to_reg_in	1-bit	WB source select
flag_write_in	1-bit	Flag update enable
is_push_in	1-bit	PUSH control
is_pop_in	1-bit	POP control
is_peak_in	1-bit	PEAK control

Output Signals

Signal Name	Width	Description
pc_out	8-bit	PC forwarded to EX stage
rs1_data_out	16-bit	Operand A for ALU
rs2_data_out	16-bit	Operand B for ALU
imm_out	16-bit	Immediate operand
rd_out	3-bit	Destination register
rs1_out	3-bit	Source register 1
rs2_out	3-bit	Source register 2
alu_op_out	3-bit	ALU operation control
alu_src_out	1-bit	ALU source control
reg_write_out	1-bit	Register write enable
mem_read_out	1-bit	Memory read enable
mem_write_out	1-bit	Memory write enable
mem_to_reg_out	1-bit	WB select control
flag_write_out	1-bit	Flag update enable
is_push_out	1-bit	PUSH control

is_pop_out	1-bit	POP control
is_peak_out	1-bit	PEAK control

Internal Working Principle

The ID/EX register captures all Decode stage outputs on every positive clock edge.

Case 1 — Reset Condition

When: **reset = 1**
all outputs become zero.
This safely initializes the Execute stage.

Case 2 — Stall Condition

When: **stall = 1**
the register holds previous values.
This prevents:

- invalid ALU execution,
- incorrect forwarding,
- and corrupted pipeline timing.

Case 3 — Normal Operation

During normal execution:

- all decoded signals are latched,
- then forwarded to the Execute stage.

This includes:

- operands,
- control signals,
- register addresses,
- and immediate values.

Importance in Processor Pipeline

The ID/EX register is extremely important because:

- Execute stage depends entirely on stable operands,
- control signals must remain synchronized,
- and ALU behavior depends on correct timing alignment.

Without this block:

- ALU operations would become unstable,
- control signals could mismatch,
- and pipelined execution would fail.

Signals Stored Inside This Register

This register stores three major categories of information.

1. Datapath Signals

- operands
- immediate values
- PC

2. Register Information

- rs1
- rs2
- rd

3. Control Signals

- ALU control
- memory control
- writeback control
- stack control

Pipeline Importance

This block forms the transition point where instruction decoding ends and actual execution begins. It is therefore one of the most timing-critical blocks in the processor.

Special Design Advantage

This implementation:

- cleanly separates datapath and control flow,
- supports hazard handling,
- integrates with forwarding logic,
- and maintains stable pipeline execution.

The design is fully synthesizable and timing-friendly.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : ID/EX Pipeline Register
// ========================================
module id_ex_reg (
    input clk,
    input reset,
    input stall,
    input flush,   // NEW

    input [7:0] pc_in,
    input [15:0] rs1_in,
    input [15:0] rs2_in,
```

```
    input [15:0] imm_in,
    input [2:0] rd_in,
    input is_push_in,
    input is_pop_in,
    input is_peak_in,

    input [2:0] alu_op_in,
    input alu_src_in,
    input reg_write_in,
    input mem_read_in,
    input mem_write_in,
    input mem_to_reg_in,
    input jump_in,
    input flag_write_in,
    input [2:0] rs1_addr_in,
    input [2:0] rs2_addr_in,
    input [1:0] group_in,
    input halt_in,
    input [2:0] opcode_in,

    output reg [2:0] opcode_out,
    output reg halt_out,
    output reg [1:0] group_out,
    output reg [2:0] rs1_addr_out,
    output reg [2:0] rs2_addr_out,
    output reg [7:0] pc_out,
    output reg [15:0] rs1_out,
    output reg [15:0] rs2_out,
    output reg [15:0] imm_out,
    output reg [2:0] rd_out,
    output reg is_push_out,
    output reg is_pop_out,
    output reg is_peak_out,

    output reg [2:0] alu_op_out,
    output reg alu_src_out,
    output reg reg_write_out,
    output reg mem_read_out,
    output reg mem_write_out,
    output reg mem_to_reg_out,
    output reg jump_out,
    output reg flag_write_out
);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        pc_out <= 0;
        rs1_out <= 0;
        rs2_out <= 0;
        imm_out <= 0;
        rd_out <= 0;
```

```
        alu_op_out <= 0;
        alu_src_out <= 0;
        reg_write_out <= 0;
        group_out <= 0;
        mem_read_out <= 0;
        mem_write_out <= 0;
        mem_to_reg_out <= 0;
        jump_out <= 0;
        halt_out <= 0;
        flag_write_out <= 0;
        rs1_addr_out <= 0;
        rs2_addr_out <= 0;
        opcode_out <= 0;
        is_push_out <= 0;
        is_pop_out  <= 0;
        is_peak_out <= 0;
    end
    else if (flush) begin
    // CORRECT NOP INSERTION
        pc_out <= pc_in;
        rs1_out <= 0;
        rs2_out <= 0;
        imm_out <= 0;
        rd_out <= 0;
        opcode_out <= 0;

        alu_op_out <= 3'b000;
        alu_src_out <= 0;
        reg_write_out <= 0;
        mem_read_out <= 0;
        mem_write_out <= 0;
        mem_to_reg_out <= 0;

        group_out <= 2'b00;   // neutral (ALU group)
        jump_out <= 0;
        halt_out <= 0;       // ❗ REMOVE HALT
        flag_write_out <= 0;

        rs1_addr_out <= 0;
        rs2_addr_out <= 0;

        is_push_out <= 0;
        is_pop_out  <= 0;
        is_peak_out <= 0;
    end
    else if (!stall) begin
        pc_out <= pc_in;
        rs1_out <= rs1_in;
        rs2_out <= rs2_in;
        imm_out <= imm_in;
        rd_out <= rd_in;
```

```
        alu_op_out <= alu_op_in;
        alu_src_out <= alu_src_in;
        reg_write_out <= reg_write_in;
        mem_read_out <= mem_read_in;
        mem_write_out <= mem_write_in;
        mem_to_reg_out <= mem_to_reg_in;
        group_out <= group_in;
        halt_out <= halt_in;
        jump_out <= jump_in;
        flag_write_out <= flag_write_in;
        rs1_addr_out <= rs1_addr_in;
        rs2_addr_out <= rs2_addr_in;
        opcode_out <= opcode_in;
        is_push_out <= is_push_in;
        is_pop_out  <= is_pop_in;
        is_peak_out <= is_peak_in;
    end
end

endmodule
```

STAGE 3 — EXECUTION STAGE (EX)

BLOCK 12 — FORWARDING UNIT

What is this Block?

The Forwarding Unit is a combinational logic block used to solve data dependency problems inside the pipelined processor without unnecessarily stalling the pipeline.
In pipelined execution, instructions overlap with each other.
Sometimes:

- a new instruction requires data,
- but that data is still being processed in later pipeline stages.

Instead of waiting for writeback completion, the Forwarding Unit directly bypasses the latest available result to the ALU inputs.
This process is called: Data Forwarding or Bypassing

Why is this Block Used?

Pipelined processors execute multiple instructions simultaneously.
Example:
ADD R5, R1, R2
SUB R6, R5, R3
The SUB instruction immediately needs: **R5**
but:

- ADD has not yet written R5 back into the Register File.

Without forwarding:

- pipeline must stall,
- performance drops,
- throughput decreases.

The Forwarding Unit solves this problem by:

- taking the latest ALU result directly from later stages,
- and feeding it back into current ALU inputs.

This avoids unnecessary stalls and significantly improves performance.

Role in the Processor

The Forwarding Unit performs the following tasks:

- Detects ALU data dependencies.
- Selects latest valid operand source.
- Controls Forward MUXes.
- Reduces pipeline stalls.
- Improves pipeline throughput.

This block works closely with:

- ALU,
- EX/MEM register,
- MEM/WB register,
- and Forward MUXes.

Key Features

1. RAW Hazard Resolution

Handles:
Read After Write (RAW)
dependencies.

2. EX Stage Priority

The newest data always gets the highest priority.

3. MEM/WB Forwarding Support

Supports forwarding from WB stage.

4. Fully Combinational Logic

No clock required.

5. High Performance Improvement

Reduces unnecessary stalls.

Input Signals

Signal Name	Width	Description
rs1_ex	3-bit	Source register 1 in EX stage
rs2_ex	3-bit	Source register 2 in EX stage
rd_mem	3-bit	Destination register in MEM stage
rd_wb	3-bit	Destination register in WB stage
reg_write_mem	1-bit	MEM stage register write enable
reg_write_wb	1-bit	WB stage register write enable

Output Signals

Signal Name	Width	Description
forwardA	2-bit	Select signal for operand A
forwardB	2-bit	Select signal for operand B

Internal Working Principle

The Forwarding Unit continuously compares:

- source registers in EX stage
  with:
- destination registers in MEM and WB stages.

EX/MEM Forwarding Check

Highest priority forwarding source.
If:
reg_write_mem = 1 AND rd_mem == rs1_ex
then:
forwardA = EX_FORWARD

Similarly:
rd_mem == rs2_ex
then:
forwardB = EX_FORWARD
This forwards the latest ALU result directly from the MEM stage.

MEM/WB Forwarding Check

If MEM-stage forwarding is not valid **AND** reg_write_wb = 1
then forwarding can occur from WB stage.
Example:
rd_wb == rs1_ex
or
rd_wb == rs2_ex

Forwarding Priority

Priority order:

Priority	Source
Highest	EX/MEM
Lower	MEM/WB

This ensures:
- newest data is always used,
- stale values are never forwarded.

Example Forwarding Scenario

Instruction 1

ADD R5, R1, R2

Instruction 2

SUB R6, R5, R3
Problem:
- SUB requires updated R5,
- but R5 is not yet written back.

Forwarding Unit detects:
rd_mem == rs1_ex
Then:
forwardA = MEM_RESULT
The latest ALU result bypasses Register File directly.

Relationship with Hazard Unit

The processor uses:
- Forwarding Unit
- Hazard Detection Unit

together.

Forwarding Unit Handles

- most ALU dependencies

Hazard Unit Handles

- load-use hazards

because memory data is not immediately available.

Importance in Pipeline

The Forwarding Unit is critical because:
- pipelined processors create many dependencies,
- stalls severely reduce performance,
- and forwarding dramatically improves throughput.

Without forwarding:
- almost every dependent instruction would stall,
- pipeline efficiency would collapse.

Special Design Advantage

This implementation:

- is lightweight,
- fast,
- synthesizable,
- and highly pipeline-efficient.

The EX-priority forwarding architecture matches real pipelined processor design techniques used in practical CPUs.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Forwarding Unit
// ========================================
module forwarding_unit(
    input [2:0] rs1_ex,
    input [2:0] rs2_ex,

    input [2:0] rd_mem,
    input [2:0] rd_wb,

    input reg_write_mem,
    input reg_write_wb,

    output reg [1:0] forwardA,
    output reg [1:0] forwardB
);

always @(*) begin
    // Default
    forwardA = 2'b00;
    forwardB = 2'b00;

    // ========================
    // FORWARD A (rs1)
    // ========================
    if (reg_write_mem && (rd_mem != 0) && (rd_mem == rs1_ex))
        forwardA = 2'b10;   // EX priority
    else if (reg_write_wb && (rd_wb == rs1_ex))
        forwardA = 2'b01;

    // ========================
    // FORWARD B (rs2)
    // ========================
```

```
    if (reg_write_mem && (rd_mem != 0) && (rd_mem == rs2_ex))
        forwardB = 2'b10;
    else if (reg_write_wb && (rd_wb == rs2_ex))
        forwardB = 2'b01;
end

endmodule
```

BLOCK 13 — FORWARD MUX A

What is this Block?

Forward MUX A is a multiplexer used in the Execute stage to select the correct source operand for ALU Operand A.

In a pipelined processor, operand data may come from:

- Register File,
- MEM stage forwarding,
- or WB stage forwarding.

This multiplexer chooses the most recent and correct value.

Why is this Block Used?

Due to pipelining, the latest result may not yet be written back into the Register File.

Example:

ADD R5, R1, R2

SUB R6, R5, R3

The SUB instruction needs the newest value of: **R5**

but the Register File still contains the old value.

Forward MUX A allows:

- forwarded ALU result,
- or WB result

to directly reach the ALU input without waiting.

Without this block:

- incorrect operands would reach ALU,
- or processor would require excessive stalls.

Role in the Processor

Forward MUX A performs the following tasks:

- Selects ALU Operand A source.
- Receives forwarding control from Forwarding Unit.
- Supports EX/MEM forwarding.
- Supports MEM/WB forwarding.

This block directly feeds: ALU Operand A

Key Features

1. Multi-Source Operand Selection

Supports:

- Register File data
- MEM forwarding
- WB forwarding

2. Hazard Resolution Support
Helps eliminate RAW hazards.

3. Fast Combinational Operation
No clock required.

4. Pipeline-Compatible
Designed specifically for pipelined execution.

5. EX Priority Support
The newest data gets the highest priority.

Input Signals

Signal Name	Width	Description
rs1_data_ex	16-bit	Operand from Register File
alu_result_mem	16-bit	Forwarded result from MEM stage
writeback_data	16-bit	Forwarded result from WB stage
forwardA	2-bit	Forwarding select signal

Output Signals

Signal Name	Width	Description
operandA	16-bit	Selected ALU operand A

Internal Working Principle
The multiplexer selects one of three inputs depending on: forwardA

Case 1 — No Forwarding
When: **forwardA = 00**
MUX selects: **rs1_data_ex**
Operand comes directly from the Register File.

Case 2 — MEM Stage Forwarding
When: **forwardA = 01**
MUX selects: **alu_result_mem**
The latest ALU result is forwarded from the MEM stage.

Case 3 — WB Stage Forwarding

When: **forwardA = 10**
MUX selects: **writeback_data**
The result is forwarded from WB stage.

Importance in Pipeline

This block is critical because:

- ALU execution depends on correct operands,
- forwarding prevents unnecessary stalls,
- and pipeline throughput improves significantly.

Without Forward MUX A:

- operand hazards would corrupt computations,
- or pipeline performance would degrade heavily.

Relationship with Forwarding Unit

The Forwarding Unit generates:
forwardA
The Forward MUX A physically applies the selection.
Together they form:
hazard bypass mechanism
inside the processor.

Special Design Advantage

This implementation:

- is lightweight,
- fast,
- synthesizable,
- and introduces very small timing overhead.

The architecture is similar to forwarding datapaths used in real pipelined CPUs.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Forward MUX A
// ========================================
module forward_mux(
    input [15:0] reg_data,
    input [15:0] ex_mem_data,
    input [15:0] mem_wb_data,
    input [1:0] forward_sel,
    output reg [15:0] out
);

always @(*) begin
```

```
    case(forward_sel)
        2'b00: out = reg_data;
        2'b10: out = ex_mem_data;
        2'b01: out = mem_wb_data;
        default: out = reg_data;
    endcase
end

endmodule
```

BLOCK 14 — FORWARD MUX B

What is this Block?

Forward MUX B is a multiplexer used in the Execute stage to select the correct source operand for ALU Operand B. This block works similarly to Forward MUX A, but it controls the second ALU operand path. Because the processor is pipelined, the latest operand value may not yet be available in the Register File.
Forward MUX B allows the processor to directly use:

- forwarded MEM-stage data,
- forwarded WB-stage data,
- or normal Register File data

depending on pipeline conditions.

Why is this Block Used?

In pipelined execution, instructions overlap.
Example:
ADD R5, R1, R2
MUL R7, R3, R5
The MUL instruction requires: **R5**
but:

- ADD has not yet completed the writeback.

Forward MUX B allows the newest R5 value to bypass directly into the ALU.
Without this block:

- ALU would receive stale data,
- computations would become incorrect,
- or pipeline stalls would increase significantly.

Role in the Processor

Forward MUX B performs the following tasks:

- Selects ALU Operand B source.
- Receives forwarding control from Forwarding Unit.
- Supports EX/MEM forwarding.
- Supports MEM/WB forwarding.
- Supplies correct operand to ALU.

This block directly feeds:
ALU Operand B

Key Features

1. Multi-Source Operand Selection

Supports:

- Register File data
- MEM-stage forwarding
- WB-stage forwarding

2. Hazard Resolution Support
Helps eliminate RAW dependencies.

3. Fast Combinational Design
No clock required.

4. Pipeline Optimization
Reduces unnecessary stalls.

5. EX-Stage Priority
Newest available operand always selected first.

Input Signals

Signal Name	Width	Description
rs2_data_ex	16-bit	Operand from Register File
alu_result_mem	16-bit	Forwarded result from MEM stage
writeback_data	16-bit	Forwarded result from WB stage
forwardB	2-bit	Forwarding select signal

Output Signals

Signal Name	Width	Description
operandB	16-bit	Selected ALU operand B

Internal Working Principle
The multiplexer selects operand source depending on: **forwardB**

Case 1 — Normal Register Operand
When: **forwardB = 00**
MUX selects: **rs2_data_ex**
Operand comes directly from the Register File.

Case 2 — MEM Stage Forwarding
When: **forwardB = 01**
MUX selects: **alu_result_mem**
The latest ALU result is forwarded directly from the MEM stage.

Case 3 — WB Stage Forwarding

When: **forwardB = 10**
MUX selects: **writeback_data**
Writeback result is forwarded from WB stage.

Importance in Pipeline

Forward MUX B is critical because:

- ALU Operand B participates in nearly every arithmetic and logic operation,
- forwarding prevents invalid execution,
- and pipeline throughput improves significantly.

Without this block:

- dependent instructions would require excessive stalls,
- or incorrect results would propagate through the pipeline.

Relationship with Forwarding Unit

The Forwarding Unit generates:
forwardB
This multiplexer implements the actual operand selection based on that signal.
Together:

- Forwarding Unit decides,
- Forward MUX B applies the selection.

Relationship with Operand MUX

Forward MUX B works before:
Operand MUX
The forwarding logic first selects the newest valid operand.
Then Operand MUX decides whether ALU should use:

- forwarded register data,
  or
- immediate value.

Pipeline Importance

This block plays a major role in maintaining:

- high instruction throughput,
- reduced stalls,
- and correct pipelined execution.

It is essential for efficient hazard handling in the Execute stage.

Special Design Advantage

This implementation:

- is lightweight,

- synthesizable,
- timing-efficient,
- and highly scalable.

The design closely follows forwarding architectures used in real pipelined processor datapaths.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Forward MUX B
// ========================================
module forward_mux(
    input [15:0] reg_data,
    input [15:0] ex_mem_data,
    input [15:0] mem_wb_data,
    input [1:0] forward_sel,
    output reg [15:0] out
);
always @(*) begin
    case(forward_sel)
        2'b00: out = reg_data;
        2'b10: out = ex_mem_data;
        2'b01: out = mem_wb_data;
        default: out = reg_data;
    endcase
end
endmodule
```

BLOCK 15 — OPERAND MUX

What is this Block?

The Operand MUX is a multiplexer used to select the second input operand for the ALU.
The ALU can operate using:

- register data,
- immediate data.

This block decides which one should be sent to the ALU based on instruction type. It acts as the selection point between register-based execution and immediate-based execution

Why is this Block Used?

Not all instructions use two registers.
Example: ADD R5, R1, R2
uses:

- two register operands.

But: LOADI R1, #5
uses:

- one register/immediate combination.

The ALU always requires: 16-bit operand inputs
Therefore the processor needs a mechanism to choose whether Operand B comes from:

- Register File,
- Immediate Generator.

The Operand MUX performs this selection.
Without this block:

- immediate instructions would fail,
- ALU inputs would mismatch,
- and instruction flexibility would reduce significantly.

Role in the Processor

The Operand MUX performs the following tasks:

- Selects ALU Operand B source.
- Chooses between register operand and immediate operand.
- Supports immediate instructions.
- Interfaces with ALU datapath.

This block directly affects:

- arithmetic operations,
- immediate operations,
- address calculations,
- and branch computations.

Key Features

1. Dual Operand Source Support

Supports:

- register operand
- immediate operand

2. Immediate Instruction Support

Enables:

- LOADI
- address calculations
- immediate ALU operations

3. Simple Combinational Design

No clock required.

4. Pipeline-Compatible

Works directly in the Execute stage.

5. Lightweight Hardware

Very small logic overhead.

Input Signals

Signal Name	Width	Description
operandB_forwarded	16-bit	Operand from Forward MUX B
imm_ex	16-bit	Immediate value from Immediate Generator
alu_src_ex	1-bit	Operand source select control

Output Signals

Signal Name	Width	Description
alu_operandB	16-bit	Final ALU Operand B

Internal Working Principle

The Operand MUX selects the ALU second operand depending on:
alu_src_ex

Case 1 — Register Operand

When: **alu_src_ex = 0**
the multiplexer selects: **operandB_forwarded**
This is used for:

- ADD
- SUB
- MUL
- DIV
- logic operations

and other register-based instructions.

Case 2 — Immediate Operand

When: **alu_src_ex = 1**
the multiplexer selects: **imm_ex**
This is used for:

- LOADI
- immediate arithmetic
- address calculations
- branch offset operations

Relationship with Forwarding Logic

Forwarding occurs BEFORE Operand MUX selection.
Flow:
Forward MUX B → Operand MUX → ALU
This ensures:

- newest operand data is selected first,
- then immediate selection is applied if needed.

Relationship with Control Unit

The Control Unit generates: alu_src_ex
This signal determines whether the ALU should use:

- register operand,
- immediate operand.

Example Operation

Register-Based Instruction

Instruction: **ADD R5, R1, R2**
Selection: **alu_src_ex = 0**
Operand B comes from: **Register File / Forwarding path**

Immediate-Based Instruction

Instruction: **LOADI R1, #5**
Selection: **alu_src_ex = 1**
Operand B comes from: **Immediate Generator**

Importance in Processor

The Operand MUX is important because:

- modern processors heavily use immediate instructions,
- ALU input flexibility is essential,
- and address generation often requires immediate values.

Without this block:

- processor functionality would become limited,
- instruction set flexibility would reduce,
- and memory operations would become harder to implement.

Special Design Advantage

This implementation:

- is very lightweight,
- synthesizes efficiently,
- introduces minimal delay,
- and integrates cleanly into pipelined datapath.

The architecture follows standard operand selection techniques used in practical processor designs.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Operand MUX
// ========================================
module operand_mux (
    input [15:0] reg_data,
    input [15:0] imm_data,
    input alu_src,
    output [15:0] operand_out
);

assign operand_out = (alu_src) ? imm_data : reg_data;

endmodule
```

BLOCK 16 — BRENT-KUNG ADDER

What is this Block?

The Brent-Kung Adder is a high-speed parallel-prefix binary adder used inside the ALU for performing fast arithmetic operations. Unlike a simple ripple-carry adder, which propagates carry sequentially from one bit to another, the Brent-Kung Adder computes carry signals in parallel using a prefix-tree structure. This significantly improves arithmetic speed.

In this processor, the Brent-Kung Adder is used as the primary arithmetic engine for:

- ADD
- SUB
- INC
- DEC
- CMP

operations.

Why is this Block Used?

Arithmetic operations are one of the most frequently executed operations inside a processor.

A traditional ripple-carry adder becomes slow because:

- each bit must wait for the previous carry,
- creating a large propagation delay.

As processor speed increases:

- arithmetic delay becomes a major timing bottleneck.

The Brent-Kung Adder solves this problem by:

- computing carries in parallel,
- reducing carry propagation delay,
- and improving ALU timing performance.

Without a fast adder:

- ALU delay increases,
- clock frequency reduces,
- and processor performance decreases.

Role in the Processor

Inside this processor, the Brent-Kung Adder performs the following functions:

- Executes high-speed addition.
- Supports subtraction operations.
- Supports increment/decrement.
- Supports compare operations.
- Generates carry information for flags.

This block forms the arithmetic backbone of the ALU.

Key Features

1. Parallel Prefix Architecture
Carry signals computed simultaneously.

2. Reduced Propagation Delay
Much faster than ripple-carry design.

3. Area-Efficient Prefix Tree
Lower hardware complexity than some other prefix adders.

4. Fast Carry Generation
Improves ALU timing performance.

5. Suitable for ASIC Design
Efficient for synthesis and physical implementation.

Input Signals

Signal Name	Width	Description
A	16-bit	First arithmetic operand
B	16-bit	Second arithmetic operand
cin	1-bit	Carry input

Output Signals

Signal Name	Width	Description
sum	16-bit	Addition/subtraction result
cout	1-bit	Final carry output

Internal Working Principle

The Brent-Kung Adder uses:
Generate and Propagate logic
to compute carry signals efficiently.

Step 1 — Generate and Propagate Calculation

For each bit:

Generate

G = A & B

Propagate

P = A ^ B

These signals determine:

- whether carry is generated,
- or propagated forward.

Step 2 — Prefix Tree Carry Computation

Carry signals are computed using:
parallel prefix network
instead of sequential ripple propagation.
This reduces carry delay significantly.

Step 3 — Sum Generation

Final sum bits are calculated using:
SUM = P ^ Carry

Subtraction Support

Subtraction is implemented using:
2's complement arithmetic
The ALU:

- inverts operand B,
- sets carry-in to 1,
- then performs addition.

Example:
A - B = A + (~B + 1)
The Brent-Kung Adder handles this efficiently.

Relationship with ALU

This block is integrated directly inside:
Pipelined ALU
The ALU uses it for:

- arithmetic computation,
- comparison,
- flag generation,
- increment/decrement.

Importance in Processor

The adder is one of the most timing-critical components in any processor.
Because arithmetic operations occur frequently:

- Adder delay strongly affects processor frequency.

Using a Brent-Kung Adder improves:

- ALU speed,
- timing closure,
- synthesis quality,
- and overall processor performance.

Why Brent-Kung Architecture Was Chosen

Compared to Ripple Carry Adder:

Feature	Ripple Carry	Brent-Kung
Speed	Slow	Fast
Carry Delay	High	Low
Parallelism	Low	High
Timing Performance	Poor	Better

Compared to larger prefix adders:

- Brent-Kung uses less area,
- lower routing complexity,
- and balanced performance.

This makes it highly suitable for:
RTL-to-GDS ASIC implementation

Pipeline Importance

The Execute stage depends heavily on fast arithmetic hardware.
A slow adder would:

- increase ALU critical path,
- reduce clock frequency,
- and negatively affect timing closure.

The Brent-Kung architecture helps maintain:

- fast execution,
- stable timing,
- and efficient synthesis.

Special Design Advantage

This implementation:

- balances speed and hardware cost,
- reduces carry propagation delay,
- synthesizes efficiently,
- and is highly suitable for ASIC physical design.

It provides significantly better performance than a conventional ripple-carry design.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Brent-Kung Adder
// ========================================
module brent_kung (
    input  [15:0] A,
    input  [15:0] B,
    input         cin,
    output [15:0] Sum,
    output        Cout
);

    wire [15:0] g0 = A & B;
    wire [15:0] p0 = A ^ B;

    wire [7:0] g1, p1;
    genvar i;
    generate
    for (i=0; i<8; i=i+1) begin : L1
        assign g1[i] = g0[2*i+1] | (p0[2*i+1] & g0[2*i]);
        assign p1[i] = p0[2*i+1] & p0[2*i];
    end
    endgenerate

    wire [3:0] g2, p2;
    generate
    for (i=0; i<4; i=i+1) begin : L2
        assign g2[i] = g1[2*i+1] | (p1[2*i+1] & g1[2*i]);
        assign p2[i] = p1[2*i+1] & p1[2*i];
    end
    endgenerate

    wire [1:0] g3, p3;
    assign g3[0] = g2[1] | (p2[1] & g2[0]);
    assign p3[0] = p2[1] & p2[0];
    assign g3[1] = g2[3] | (p2[3] & g2[2]);
    assign p3[1] = p2[3] & p2[2];

    wire g4, p4;
    assign g4 = g3[1] | (p3[1] & g3[0]);
```

```
    assign p4 = p3[1] & p3[0];

    wire [16:0] C;
    assign C[0] = cin;

    assign C[1]  = g0[0]  | (p0[0]  & C[0]);
    assign C[2]  = g1[0]  | (p1[0]  & C[0]);
    assign C[4]  = g2[0]  | (p2[0]  & C[0]);
    assign C[8]  = g3[0]  | (p3[0]  & C[0]);
    assign C[16] = g4     | (p4     & C[0]);

    assign C[3]  = g0[2]  | (p0[2]  & C[2]);
    assign C[5]  = g0[4]  | (p0[4]  & C[4]);
    assign C[6]  = g1[2]  | (p1[2]  & C[4]);
    assign C[7]  = g0[6]  | (p0[6]  & C[6]);
    assign C[9]  = g0[8]  | (p0[8]  & C[8]);
    assign C[10] = g1[4]  | (p1[4]  & C[8]);
    assign C[11] = g0[10] | (p0[10] & C[10]);
    assign C[12] = g2[2]  | (p2[2]  & C[8]);
    assign C[13] = g0[12] | (p0[12] & C[12]);
    assign C[14] = g1[6]  | (p1[6]  & C[12]);
    assign C[15] = g0[14] | (p0[14] & C[14]);

    assign Sum  = p0 ^ C[15:0];
    assign Cout = C[16];
endmodule
```

BLOCK 17 — SHIFT-ADD MULTIPLIER

What is this Block?

The Shift-Add Multiplier is a multi-cycle arithmetic hardware block used to perform binary multiplication inside the processor. Instead of using a very large combinational multiplier circuit, this design performs multiplication iteratively using:

- shifting,
- addition,
- and accumulation.

The multiplication operation is completed across multiple clock cycles.
This block is used for executing:
MUL
instructions.

Why is this Block Used?

Multiplication is a computationally expensive operation.
A fully combinational multiplier:

- consumes large hardware area,
- increases routing complexity,
- and creates a large timing delay.

To reduce hardware complexity, this processor uses: Shift-Add Multiplication
which performs multiplication gradually over several cycles.

This approach:

- reduces area usage,
- simplifies synthesis,
- and improves ASIC implementation feasibility.

Without a dedicated multiplier:

- MUL instructions would not be supported,
- arithmetic capability would reduce,
- and processor functionality would become incomplete.

Role in the Processor

The Shift-Add Multiplier performs the following functions:

- Executes MUL instruction.
- Multiplies two 16-bit operands.
- Generates 16-bit multiplication results.
- Supports multi-cycle execution.
- Generates busy/done status signals.

This block operates as part of the ALU execution subsystem.

Key Features

1. Multi-Cycle Operation
Computation spread across multiple clock cycles.

2. Area-Efficient Architecture
Uses significantly less hardware than a combinational multiplier.

3. Shift-and-Add Algorithm
Simple and synthesizable design.

4. Busy/Done Control
Supports pipeline stall handling.

5. ASIC-Friendly Design
Reduces routing and timing complexity.

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock
reset	1-bit	Resets multiplier
start	1-bit	Starts multiplication
multiplicand	16-bit	First multiplication operand
multiplier	16-bit	Second multiplication operand

Output Signals

Signal Name	Width	Description
product	16-bit	Final multiplication result
busy	1-bit	Indicates multiplication in progress
done	1-bit	Indicates multiplication completed

Internal Working Principle

The multiplier uses the classical: Shift-and-Add Algorithm
for binary multiplication.

Step 1 — Initialization

When: **start = 1**
the multiplier:

- loads operands,
- clears accumulator,
- sets a busy signal.

Step 2 — Check Multiplier LSB

The multiplier checks: **multiplier[0]**

If LSB = 1

Add multiplicand to the accumulator.

If LSB = 0

No addition performed.

Step 3 — Shift Operation

After each cycle:

- multiplicand shifts left,
- multiplier shifts right.

This aligns partial products correctly.

Step 4 — Repeat Iteratively

The process continues until all multiplier bits processed

Step 5 — Completion

When multiplication finishes: **done = 1 and busy = 0**
Final result becomes available on: **product**

Example Multiplication

Example: **5 × 3**
Binary: **0101 × 0011**
Operations:

- Add shifted multiplicand when multiplier bit = 1
- Shift operands each cycle
- Accumulate partial sums

Final result: **15**

Multi-Cycle Execution

Unlike normal ALU operations:

- multiplication cannot complete in one cycle.

During multiplication: **busy = 1**
This signal:

- stalls pipeline,
- prevents incorrect execution,
- and synchronizes ALU timing.

Relationship with Stall Unit

The multiplier connects directly with Stall Unit
When: **busy = 1**
pipeline execution pauses until multiplication completes.
This prevents:

- operand corruption,
- invalid forwarding,
- and timing mismatches.

Importance in Processor

Multiplication support is important because:

- many arithmetic applications require multiplication,
- DSP-style operations depend on it,
- and processor computational capability improves significantly.

The multiplier extends processor functionality beyond basic arithmetic.

Why Multi-Cycle Design Was Chosen

Compared to combinational multiplier:

Feature	Combinational	Shift-Add
Hardware Area	Large	Small
Timing Delay	High	Lower
Routing Complexity	High	Low
ASIC Friendliness	Moderate	Better

The multi-cycle design is more suitable for:
student ASIC RTL-to-GDS implementation
because it:

- simplifies backend design,
- improves synthesis,
- and reduces critical path complexity.

Pipeline Importance

The processor pipeline must coordinate carefully with multi-cycle units.
This multiplier demonstrates:

- pipeline stall handling,
- execution synchronization,
- and multi-cycle operation integration.

It is one of the most advanced execution blocks in the processor.

Special Design Advantage

This implementation:

- minimizes hardware overhead,
- integrates cleanly with pipeline,
- supports ASIC synthesis efficiently,
- and demonstrates practical multi-cycle processor execution.

The architecture is highly educational and hardware-efficient.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Shift-Add Multiplier
// ========================================
module mul_unit (
    input  wire        clk,
    input  wire        rst,
    input  wire        start,
    input  wire [15:0] A1,
    input  wire [15:0] B1,
    output reg         mul_busy,
    output reg         mul_done,
    output reg  [15:0] mul_prod
);

    reg [4:0]  mul_cnt;
    reg [15:0] mul_m, mul_q;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            mul_busy <= 0;
            mul_done <= 0;
            mul_cnt  <= 0;
            mul_prod <= 0;
            mul_m    <= 0;
            mul_q    <= 0;
        end
        else begin
            mul_done <= 0;
```

```
        if (!mul_busy && start) begin
            mul_busy <= 1;
            mul_cnt  <= 16;
            mul_prod <= 0;
            mul_m    <= A1;
            mul_q    <= B1;
        end
        else if (mul_busy) begin
            if (mul_q[0])
                mul_prod <= mul_prod + mul_m;

            mul_m   <= mul_m << 1;
            mul_q   <= mul_q >> 1;
            mul_cnt <= mul_cnt - 1;

            if (mul_cnt == 1) begin
                mul_busy <= 0;
                mul_done <= 1;
            end
        end
    end
  end
endmodule
```

BLOCK 18 — RESTORING DIVIDER

What is this Block?

The Restoring Divider is a multi-cycle arithmetic hardware block used to perform binary division inside the processor. This block executes: DIV
instructions using the: Restoring Division Algorithm
Instead of using a large combinational divider circuit, division is performed gradually over multiple clock cycles using:

- shifting,
- subtraction,
- comparison,
- and remainder restoration.

This reduces hardware complexity and improves synthesizability.

Why is this Block Used?

Division is one of the most computationally expensive arithmetic operations in digital hardware.
A fully combinational divider:

- consumes very large hardware area,
- creates long critical paths,
- and is difficult to optimize physically.

To avoid these problems, this processor uses: iterative restoring division
which performs division step-by-step across multiple cycles.
This approach:

- reduces hardware overhead,
- improves timing closure,
- and simplifies ASIC implementation.

Without a divider:

- DIV instruction would not exist,
- processor arithmetic capability would be incomplete,
- and computational flexibility would reduce significantly.

Role in the Processor

The Restoring Divider performs the following tasks:

- Executes DIV instruction.
- Divides two 16-bit operands.
- Generates quotient results.
- Handles divide-by-zero conditions.
- Supports multi-cycle execution.
- Generates busy/done synchronization signals.

This block is integrated directly into the ALU execution subsystem.

Key Features

1. Multi-Cycle Operation
Division completes over several clock cycles.

2. Restoring Division Algorithm
Uses iterative subtraction and restoration.

3. Busy/Done Synchronization
Supports pipeline stall management.

4. Divide-by-Zero Protection
Prevents invalid arithmetic behavior.

5. Area-Efficient Hardware
Uses much less hardware than a combinational divider.

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock
reset	1-bit	Resets divider
start	1-bit	Starts division operation
dividend	16-bit	Dividend operand
divisor	16-bit	Divisor operand

Output Signals

Signal Name	Width	Description
quotient	16-bit	Final division result
busy	1-bit	Indicates division in progress
done	1-bit	Indicates division completed
divide_by_zero	1-bit	Indicates divisor is zero

Internal Working Principle

The divider uses the classical: Restoring Division Algorithm
to compute division iteratively.

Step 1 — Initialization

When: **start = 1**
the divider:

- loads dividend,
- loads divisor,
- clears quotient,
- initializes remainder,
- sets a busy signal.

Step 2 — Shift Operation

The partial remainder shifts left by one bit.
The next dividend bit is inserted.
This aligns operands for subtraction.

Step 3 — Subtraction

The divider subtracts divisor from partial remainder

Step 4 — Result Check

If subtraction is positive:

- quotient bit becomes `1`
- subtraction result is retained

If subtraction becomes negative:

- quotient bit becomes `0`
- old remainder is restored

This is why the algorithm is called:
Restoring Division

Step 5 — Repeat Iteratively

The process continues bit-by-bit until:
all quotient bits generated

Step 6 — Completion

When division finishes: **done = 1 and busy = 0**
Final quotient becomes available on: **quotient**

Divide-by-Zero Protection

The divider includes safety detection logic.
If: **divisor == 0**
then: **divide_by_zero = 1**
This prevents invalid arithmetic behavior.
The operation safely terminates without corrupting processor execution.

Example Division

Example: **10 ÷ 2**
Binary: **1010 ÷ 0010**
Operations:

- shift remainder,
- subtract divisor,
- restore if negative,
- generate quotients bit-by-bit.

Final quotient: **5**

Multi-Cycle Execution

Division requires multiple cycles to complete.
During execution: **busy = 1**
This signal:

- stalls pipeline,
- pauses instruction advancement,
- and prevents execution conflicts.

When operation finishes**: done = 1**
pipeline execution resumes.

Relationship with Stall Unit

The divider interacts directly with:
Stall Unit
During division:

- pipeline execution pauses,
- PC and pipeline registers hold values,
- and synchronization is maintained.

This prevents:

- invalid operand usage,
- forwarding conflicts,
- and execution corruption.

Relationship with ALU

The Restoring Divider operates as a dedicated arithmetic submodule inside:
Pipelined ALU
The ALU:

- starts division,
- waits for completion,
- and receives quotient results after a signal.

Importance in Processor

Division support significantly improves processor capability.
Many applications require:

- scaling,
- arithmetic computation,
- ratio calculation,
- and mathematical processing.

Supporting DIV instruction makes the processor much more complete and realistic.

Why Multi-Cycle Divider Was Chosen

Compared to combinational divider:

Feature	Combinational Divider	Restoring Divider
Hardware Area	Very Large	Small
Timing Complexity	High	Lower
Routing Difficulty	High	Moderate
ASIC Friendliness	Difficult	Better

The restoring divider is more suitable for: RTL-to-GDS ASIC implementation
because it:

- reduces hardware complexity,
- improves synthesis quality,
- and lowers critical path pressure.

Pipeline Importance

The divider demonstrates advanced processor capabilities including:

- multi-cycle execution,
- stall synchronization,
- execution control,
- and iterative arithmetic hardware integration.

This makes the processor architecture significantly more realistic.

Special Design Advantage

This implementation:

- minimizes hardware overhead,
- supports reliable division,
- integrates cleanly into pipeline,
- and remains highly synthesizable.

The design demonstrates practical hardware-efficient arithmetic implementation techniques used in real processor architectures.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Restoring Divider
// ========================================
module div_unit (
    input  wire        clk,
    input  wire        rst,
    input  wire        start,
    input  wire [15:0] A1,
    input  wire [15:0] B1,
    output reg         div_busy,
    output reg         div_done,
    output reg  [15:0] div_q
);

    reg [16:0] div_r;
    reg [15:0] div_m;
    reg [4:0]  div_cnt;
    reg        div_active;   // prevents restart

    reg [16:0] next_r;
    reg [15:0] next_q;

    always @(*) begin
        next_r = div_r;
        next_q = div_q;

        if (div_busy) begin
            next_r = {div_r[15:0], div_q[15]};
            next_q = {div_q[14:0], 1'b0};

            next_r = next_r - {1'b0, div_m};

            if (next_r[16]) begin
                next_r = next_r + {1'b0, div_m};
                next_q[0] = 0;
            end else begin
                next_q[0] = 1;
            end
```

```
    end
  end

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      div_busy   <= 0;
      div_done   <= 0;
      div_q      <= 0;
      div_r      <= 0;
      div_m      <= 0;
      div_cnt    <= 0;
      div_active <= 0;
    end else begin
      div_done <= 0;

      // Start only once per instruction
      if (start && !div_active) begin
        div_active <= 1;

        if (B1 == 0) begin
          div_q    <= 16'hFFFF;
          div_done <= 1;
          div_busy <= 0;
        end else begin
          div_busy <= 1;
          div_cnt  <= 15;
          div_q    <= A1;
          div_r    <= 0;
          div_m    <= B1;
        end
      end
      else if (div_busy) begin
        div_r <= next_r;
        div_q <= next_q;

        if (div_cnt == 0) begin
          div_busy <= 0;
          div_done <= 1;
        end else begin
          div_cnt <= div_cnt - 1;
        end
      end

      // Clear active when instruction leaves EX
      if (!start)
        div_active <= 0;
    end
  end
endmodule
```

BLOCK 19 — PIPELINED ALU (ARITHMETIC LOGIC UNIT)

What is this Block?

The Arithmetic Logic Unit (ALU) is the main computational engine of the processor.
It is the block responsible for performing:

- arithmetic operations,
- logical operations,
- shift operations,
- rotate operations,
- comparison operations,
- multiplication,
- and division.

Almost every instruction executed by the processor passes through the ALU.

This processor uses a: Pipelined Multi-Operation ALU
that integrates:

- Brent-Kung Adder,
- Shift-Add Multiplier,
- Restoring Divider,
- flag generation logic,
- and execution control logic.

The ALU is one of the most important and most complex blocks in the processor.

Why is this Block Used?

A processor must perform computations on data.
These computations include:

- addition,
- subtraction,
- logic operations,
- comparisons,
- shifting,
- and arithmetic processing.

The ALU performs all these operations.

Without the ALU:

- no mathematical computation is possible,
- no logical processing is possible,
- and the processor cannot execute meaningful instructions.

Role in the Processor

The ALU performs the following functions:

- Executes arithmetic instructions.
- Executes logical instructions.
- Generates condition flags.
- Supports multi-cycle multiplication and division.
- Produces execution results for Writeback stage.
- Supports branch comparison logic.

This block forms the core execution engine of the processor.

Supported Operations

Arithmetic Operations

- ADD
- SUB
- INC
- DEC
- CMP
- ABS

Logical Operations

- AND
- OR
- XOR
- NOT

Shift Operations

- SHL
- SHR

Rotate Operations

- ROL
- ROR

Multi-Cycle Operations

- MUL
- DIV

Key Features

1. Multi-Operation Support

Supports complete arithmetic and logic instruction set.

2. Fast Brent-Kung Adder
Improves arithmetic timing performance.

3. Multi-Cycle MUL/DIV
Supports advanced arithmetic operations.

4. NZCV Flag Generation
Generates processor condition flags.

5. Pipeline-Compatible Architecture
Fully integrated into the Execute stage.

6. Stall Generation
Supports pipeline synchronization during multi-cycle operations.

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock
reset	1-bit	Resets ALU state
A	16-bit	Operand A
B	16-bit	Operand B
alu_op	3-bit	ALU operation select
start_mul	1-bit	Starts multiplication
start_div	1-bit	Starts division

Output Signals

Signal Name	Width	Description
result	16-bit	Final ALU result
N	1-bit	Negative flag
Z	1-bit	Zero flag
C	1-bit	Carry flag

V	1-bit	Overflow flag
busy	1-bit	Indicates ALU busy
done	1-bit	Indicates operation completion

Internal Architecture

The ALU internally integrates multiple arithmetic sub-blocks.

Brent-Kung Adder

Used for:

- ADD
- SUB
- INC
- DEC
- CMP

Logic Unit

Used for:

- AND
- OR
- XOR
- NOT

Shift/Rotate Unit

Used for:

- SHL
- SHR
- ROL
- ROR

Shift-Add Multiplier

Used for:

- MUL

Restoring Divider

Used for:

- DIV

Internal Working Principle

The ALU operation depends on: **alu_op**
Control signal generated by Control Unit.

Arithmetic Operations

Example:
ADD
SUB
INC
DEC
These operations use: **Brent-Kung Adder**
for fast arithmetic execution.

Logical Operations

Example:
AND
OR
XOR
NOT
These are implemented using combinational bitwise logic.

Shift and Rotate Operations

Shift Left

SHL

Shift Right

SHR

Rotate Left

ROL

Rotate Right

ROR
These manipulate bit positions directly.

Multiplication Operation

When: MUL instruction detected
the ALU:

- starts Shift-Add Multiplier,
- asserts busy signal,
- waits for done signal,
- then outputs a multiplication result.

Division Operation

When: DIV instruction detected
the ALU:

- starts Restoring Divider,
- stalls pipeline,
- waits for division completion,
- outputs quotient results.

NZCV Flag Generation

The ALU generates four processor status flags.

Flag	Meaning
N	Result is negative
Z	Result is zero
C	Carry generated
V	Arithmetic overflow

Flag Usage

Flags are used by:

- Branch Unit
- CMP instruction
- Conditional branch instructions

Example:
JZ
JNZ
JC
depend directly on ALU-generated flags.

Multi-Cycle Stall Handling

For:

- MUL
- DIV

the ALU asserts:
busy = 1
This signal:

- stalls pipeline,
- prevents invalid execution,
- synchronizes multi-cycle operations.

Relationship with Pipeline

The ALU is located entirely inside:
Execute Stage (EX)
It receives:

- operands from forwarding path,
- immediate values from Operand MUX,
- control signals from the Control Unit.

It outputs:

- execution result,
- status flags,
- and stall conditions.

Importance in Processor

The ALU is the computational heart of the processor.
Every major instruction depends on this block.
Incorrect ALU behavior would corrupt:

- arithmetic computation,
- logical processing,
- memory addressing,
- branch execution,
- and processor state.

Why This ALU Design is Strong

This ALU demonstrates:

- advanced datapath integration,
- fast arithmetic design,
- multi-cycle execution handling,
- pipeline synchronization,
- and realistic processor architecture implementation.

It is significantly more advanced than simple educational single-cycle ALUs.

Special Design Advantage

This implementation:

- balances performance and hardware cost,
- integrates multiple arithmetic units efficiently,
- supports ASIC-friendly synthesis,
- and demonstrates realistic pipelined execution techniques.

The architecture is highly suitable for:
RTL-to-GDS processor implementation
and advanced VLSI project demonstration.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Pipelined ALU
// ========================================
module pipelined_alu(
    input  wire        clk,
    input  wire        rst,
    input  wire [15:0] A,
    input  wire [15:0] B,
    input  wire [4:0]  opcode,
    output reg [3:0]  flags,
    output reg  [15:0] result,
    output wire        stall
);

    reg [15:0] A1, B1;
    reg [4:0]  opcode_r;

    wire mul_busy, mul_done;
    wire div_busy, div_done;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            A1 <= 0;
            B1 <= 0;
            opcode_r <= 0;
        end
        else if (!stall) begin
            A1 <= A;
            B1 <= B;
            opcode_r <= opcode;
        end
    end

    wire [1:0] group = opcode_r[4:3];
    wire [2:0] func  = opcode_r[2:0];

    wire is_alu   = (group == 2'b00);
    wire is_logic = (group == 2'b01);

    wire is_add = is_alu && (func == 3'b000);
    wire is_sub = is_alu && (func == 3'b001);
    wire is_mul = is_alu && (func == 3'b010);
    wire is_div = is_alu && (func == 3'b011);
    wire is_inc = is_alu && (func == 3'b100);
    wire is_dec = is_alu && (func == 3'b101);
    wire is_cmp = is_alu && (func == 3'b110);
    wire is_abs = is_alu && (func == 3'b111);

    wire is_and = is_logic && (func == 3'b000);
```

```
wire is_or  = is_logic && (func == 3'b001);
wire is_not = is_logic && (func == 3'b010);
wire is_xor = is_logic && (func == 3'b011);
wire is_shl = is_logic && (func == 3'b100);
wire is_shr = is_logic && (func == 3'b101);
wire is_ror = is_logic && (func == 3'b110);
wire is_rol = is_logic && (func == 3'b111);

assign stall = mul_busy | div_busy;

wire is_subtract = is_sub || is_cmp;

wire [15:0] adder_B =
    is_subtract ? ~B1 :
    is_inc ? 16'd1 :
    is_dec ? 16'hFFFF :
    B1;

wire adder_cin =
    is_subtract ? 1'b1 :
    1'b0;

wire [15:0] add_res;
wire add_cout;

brent_kung ADDER (
    .A(A1),
    .B(adder_B),
    .cin(adder_cin),
    .Sum(add_res),
    .Cout(add_cout)
);

wire overflow_add =
( A1[15] &  adder_B[15] & ~add_res[15]) |
(~A1[15] & ~adder_B[15] &  add_res[15]);

wire overflow_sub =
( A1[15] & ~B1[15] & ~add_res[15]) |
(~A1[15] &  B1[15] &  add_res[15]);

wire V_flag =
(is_add || is_inc) ? overflow_add :
(is_sub || is_dec || is_cmp) ? overflow_sub :
1'b0;

wire [15:0] logic_res =
    is_and ? (A1 & B1) :
    is_or  ? (A1 | B1) :
    is_xor ? (A1 ^ B1) :
    is_not ? (~A1) :
```

```
    is_shl ? (A1 << 1) :
    is_shr ? (A1 >> 1) :
    is_ror ? {A1[0], A1[15:1]} :
    is_rol ? {A1[14:0], A1[15]} :
    16'b0;

wire [15:0] abs_res = A1[15] ? (~A1 + 1) : A1;
wire [15:0] mul_prod;
wire [15:0] div_q;

reg is_div_d, is_mul_d;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        is_div_d <= 0;
        is_mul_d <= 0;
    end else if (!stall) begin
        is_div_d <= is_div;
        is_mul_d <= is_mul;
    end
end

wire start_div = is_div & ~is_div_d;
wire start_mul = is_mul & ~is_mul_d;

mul_unit MUL (
    .clk(clk),
    .rst(rst),
    .start(start_mul),
    .A1(A1),
    .B1(B1),
    .mul_busy(mul_busy),
    .mul_done(mul_done),
    .mul_prod(mul_prod)
);
div_unit DIV (
    .clk(clk),
    .rst(rst),
    .start(start_div),
    .A1(A1),
    .B1(B1),
    .div_busy(div_busy),
    .div_done(div_done),
    .div_q(div_q)
);

always @(posedge clk or posedge rst) begin
    if (rst) begin
        result <= 0;
        flags  <= 0;
    end
```

```
        // MULTI-CYCLE RESULTS HAVE TOP PRIORITY
        else if (mul_done) begin
            result <= mul_prod;
            flags[3] <= mul_prod[15];
            flags[2] <= (mul_prod == 0);
            flags[1] <= 0;
            flags[0] <= 0;
        end

        else if (div_done) begin
            result <= div_q;
            flags[3] <= div_q[15];
            flags[2] <= (div_q == 0);
            flags[1] <= 0;
            flags[0] <= 0;
        end

        // ONLY EXECUTE NORMAL ALU WHEN NOT STALLED
        else if (!stall) begin
            if (is_add || is_sub || is_inc || is_dec) begin
                result <= add_res;
                flags[3] <= add_res[15];
                flags[2] <= (add_res == 0);
                flags[1] <= add_cout;
                flags[0] <= V_flag;
            end
            else if (is_cmp) begin
                result <= add_res;   // or 0, but must assign
                flags[3] <= add_res[15];
                flags[2] <= (add_res == 0);
                flags[1] <= add_cout;
                flags[0] <= V_flag;
            end
            else if (is_logic) begin
                result <= logic_res;
                flags[3] <= logic_res[15];
                flags[2] <= (logic_res == 0);
                flags[1] <= 0;
                flags[0] <= 0;
            end
            else if (is_abs) begin
                result <= abs_res;
                flags[3] <= abs_res[15];        // N
                flags[2] <= (abs_res == 0);     // Z
                flags[1] <= 0;                  // C
                flags[0] <= (A1 == 16'h8000);  // V (overflow for -32768)
            end
        end
    end
endmodule
```

BLOCK 20 — FLAG REGISTER

What is this Block?

The Flag Register is a sequential storage block used to store processor condition flags generated by the ALU after arithmetic and logical operations. These flags describe the status of the last computation performed by the processor. In this processor, the Flag Register stores:

- Negative flag (N)
- Zero flag (Z)
- Carry flag (C)
- Overflow flag (V)

Together these are called: NZCV Flags

These flags are essential for:

- comparisons,
- conditional branching,
- arithmetic status checking,
- and processor decision making.

Why is this Block Used?

The processor often needs to make decisions based on previous computation results.
Example: CMP R1, R2
JZ label
The processor must know:

- whether result became zero,
- whether overflow occurred,
- whether carry was generated,
- or whether the result became negative.

The Flag Register stores this information permanently until the next valid update.

Without the Flag Register:

- conditional branches would fail,
- comparison instructions would become useless,
- and processor decision-making capability would break.

Role in the Processor

The Flag Register performs the following tasks:

- Stores ALU status flags.
- Maintains processor condition state.
- Supplies branch conditions to Branch Unit.
- Preserves arithmetic status across pipeline stages.

This block forms the status-tracking mechanism of the processor.

Key Features

1. NZCV Flag Storage

Stores four important processor condition flags.

2. Sequential Update

Flags update only on clock edge.

3. Controlled Flag Writing

Flags update only when enabled

.

4. Branch-Compatible

Works directly with conditional branch logic.

Flags Stored

Flag	Meaning
N	Negative Result Flag
Z	Zero Result Flag
C	Carry Flag
V	Overflow Flag

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock
reset	1-bit	Clears all flags
flag_write	1-bit	Enables flag update
N_in	1-bit	Negative flag input
Z_in	1-bit	Zero flag input
C_in	1-bit	Carry flag input
V_in	1-bit	Overflow flag input

Output Signals

Signal Name	Width	Description
N	1-bit	Stored Negative flag
Z	1-bit	Stored Zero flag
C	1-bit	Stored Carry flag
V	1-bit	Stored Overflow flag

Internal Working Principle

The Flag Register updates on every positive clock edge.

Case 1 — Reset Condition

When: **reset = 1**
all flags become: **0**
This initializes processor condition state safely.

Case 2 — Flag Update Enabled

When: **flag_write = 1**
the register stores:

- ALU-generated flags,
- produced by current instruction.

Example:
N <= N_in;
Z <= Z_in;
C <= C_in;
V <= V_in;

Case 3 — No Flag Update

When: **flag_write = 0**
previous flag values are preserved.
This is important because:

- not all instructions should modify flags,
- memory operations usually preserve status flags.

Flag Generation Examples

Zero Flag (Z)

Set when**: result == 0**
Used for: **JZ instruction.**

Negative Flag (N)

Set when: **result[15] = 1**
indicating negative signed results.

Carry Flag (C)

Set when arithmetic carry occurs during:

- addition,
- subtraction,
- or shift operations.

Overflow Flag (V)

Set when signed arithmetic overflow occurs.
Example: **32767 + 1**
causes overflow.

Relationship with ALU

The ALU generates:

- N
- Z
- C
- V

signals after execution.
The Flag Register stores these values for future instructions.

Relationship with Branch Unit

Conditional branch instructions depend heavily on flags.
Examples:

Instruction	Flag Used
JZ	Z
JNZ	Z
JC	C
JN	N

The Branch Unit reads Flag Register outputs to determine:
whether branch should occur

Pipeline Importance

The Flag Register is critical because:

- conditional execution depends on it,

- branch decisions require stable flags,
- and arithmetic status must persist across cycles.

Incorrect flag storage would break:

- comparisons,
- branching,
- and processor control flow.

Special Design Advantage

This implementation:

- cleanly separates flag storage from ALU,
- improves pipeline synchronization,
- simplifies branch logic,
- and remains lightweight and synthesizable.

The architecture follows standard processor status-register design principles used in practical CPUs.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Flag Register
// ========================================
module flag_register (
    input clk,
    input reset,
    input flag_write,
    input [3:0] flags_in,
    output reg [3:0] flags_out
);

always @(posedge clk or posedge reset) begin
    if (reset)
        flags_out <= 0;
    else if (flag_write)
        flags_out <= flags_in;
end

endmodule
```

BLOCK 21 — BRANCH UNIT

What is this Block?

The Branch Unit is a combinational control logic block responsible for deciding whether the processor should continue sequential execution or jump to a different instruction address.
It controls the processor's execution flow during:

- branch instructions,
- jump instructions,
- and conditional execution.

The Branch Unit uses:

- instruction control signals,
- branch targets,
- and processor flags

to determine whether a branch should occur.

Why is this Block Used?

Normally, processors execute instructions sequentially:
PC = PC + 1
However, programs often require:

- loops,
- conditional execution,
- decision-making,
- and jumps.

Example:

```
if (A == B)
  jump
```

The Branch Unit allows the processor to:

- change execution flow,
- repeat loops,
- skip instructions,
- and implement conditional logic.

Without this block:

- loops would not work,
- conditions could not be evaluated,
- and intelligent program execution would become impossible.

Role in the Processor

The Branch Unit performs the following tasks:

- Evaluates branch conditions.

- Reads processor flags.
- Determines branch decisions.
- Generates branch_taken signal.
- Generates branch target address.

This block directly controls:
Program Counter redirection
inside the pipeline.

Key Features

1. Conditional Branch Support

Supports:

- JZ
- JNZ
- JC
- JN

2. Unconditional Jump Support

Supports:

- JMP

3. Flag-Based Decision Logic

Uses NZCV flags for condition evaluation.

4. Fast Combinational Design

No clock required.

5. Pipeline Flush Integration

Works with Flush Unit to remove wrong-path instructions.

Input Signals

Signal Name	Width	Description
jump	1-bit	Indicates branch/jump instruction
opcode	3-bit	Branch operation type
pc	8-bit	Current Program Counter
imm	5-bit	Branch offset/immediate
N	1-bit	Negative flag

Z	1-bit	Zero flag
C	1-bit	Carry flag
V	1-bit	Overflow flag

Output Signals

Signal Name	Width	Description
branch_taken	1-bit	Indicates branch should occur
branch_addr	8-bit	Branch target address

Supported Branch Instructions

Instruction	Condition
JMP	Always branch
JZ	Branch if Z = 1
JNZ	Branch if Z = 0
JC	Branch if C = 1
JN	Branch if N = 1

Internal Working Principle

The Branch Unit continuously evaluates:

- branch opcode,
- branch enable signal,
- and processor flags.

Unconditional Jump

For:
JMP
the branch always occurs.
branch_taken = 1

Zero Branch

For:
JZ

branch occurs only if:
Z = 1
meaning the previous result was zero.

Non-Zero Branch

For:
JNZ
branch occurs when:
Z = 0

Carry Branch

For:
JC
branch occurs if:
C = 1

Negative Branch

For:
JN
branch occurs if:
N = 1

Branch Address Generation

The Branch Unit calculates target address using:
pc + imm
This generates:
branch_addr
which is later loaded into the Program Counter.

Example Operation

Example 1 — JZ

Instruction:
JZ +3
Condition:
Z = 1
Then:
branch_taken = 1
branch_addr = pc + 3
The processor jumps forward by 3 instructions.

Example 2 — JMP

Instruction:
JMP -2
Branch always occurs.
The processor jumps backward to create loop behavior.

Relationship with Flag Register

The Branch Unit directly depends on:
NZCV flags
stored inside Flag Register.
These flags determine:

- whether conditions are true,
- and whether branch execution should occur.

Relationship with Flush Unit

When:
branch_taken = 1
the Flush Unit:

- removes incorrect instructions,
- and corrects pipeline execution flow.

Relationship with Program Counter

The Branch Unit generates:
branch_addr
which becomes the next PC value during branch execution.
This redirects processor control flow.

Pipeline Importance

Branch handling is one of the most important aspects of pipelined processor design.
Incorrect branch handling can cause:

- wrong instruction execution,
- pipeline corruption,
- invalid control flow,
- and unpredictable processor behavior.

The Branch Unit ensures:

- safe control flow,
- correct branching,
- and reliable pipeline operation.

Special Design Advantage

This implementation:

- is lightweight,
- fast,
- synthesizable,
- and highly pipeline-compatible.

The branch logic integrates cleanly with:

- PC update logic,
- Flush Unit,
- and Flag Register.

This creates a robust control-flow system for the processor.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Branch Unit
// ========================================
module branch_unit (
    input jump,
    input [4:0] opcode,
    input [3:0] flags_out,   // FIXED
    input [7:0] pc_ex,
    input [7:0] imm_ex,
    output reg branch_taken,
    output reg [7:0] branch_addr
);

wire [1:0] group = opcode[4:3];
wire [2:0] func  = opcode[2:0];

wire N = flags_out[3];
wire Z = flags_out[2];
wire C = flags_out[1];
wire V = flags_out[0];

always @(*) begin
    branch_taken = 0;

    if (jump)
        branch_taken = 1;
    else if (group == 2'b11 && func == 3'b001 && Z)
        branch_taken = 1;
    else if (group == 2'b11 && func == 3'b010 && !Z)
        branch_taken = 1;
    else if (group == 2'b11 && func == 3'b011 && C)
        branch_taken = 1;
    else if (group == 2'b11 && func == 3'b100 && !C)
```

```
        branch_taken = 1;
    else if (group == 2'b11 && func == 3'b101 && N)
        branch_taken = 1;
    else if (group == 2'b11 && func == 3'b110 && !N)
        branch_taken = 1;

    if (branch_taken)
        branch_addr = (pc_ex + imm_ex) & 8'hFF;
    else
        branch_addr = pc_ex;   // safe hold
end

endmodule
```

BLOCK 22 — STALL UNIT

What is this Block?

The Stall Unit is a control logic block responsible for temporarily pausing pipeline execution whenever the processor detects conditions that could lead to incorrect execution.
A pipeline stall means:
pipeline advancement temporarily stops
while current operations complete safely.
The Stall Unit combines stall requests coming from:

- Hazard Detection Unit
- Multiplier
- Divider

and generates a global pipeline stall signal.

Why is this Block Used?

In a pipelined processor, multiple instructions execute simultaneously.
Sometimes execution must pause because:

- required data is not ready,
- multi-cycle operations are still running,
- or hazards exist between instructions.

Example:
LOAD R3, [R1]
ADD R4, R3, R2
The ADD instruction cannot execute immediately because:
R3
is not yet available.
Similarly:

- multiplication,
- and division

take multiple cycles to complete.

The Stall Unit prevents:

- invalid execution,
- corrupted operands,
- incorrect forwarding,
- and pipeline timing errors.

Without this block:

- pipeline synchronization would fail,
- execution would become unstable,
- and processor results would become incorrect.

Role in the Processor

The Stall Unit performs the following tasks:

- Receives stall requests from execution units.
- Generates global pipeline stall signal.
- Freezes Program Counter.
- Freezes pipeline registers.
- Synchronizes multi-cycle execution.

This block acts as the central execution synchronization controller.

Key Features

1. Global Stall Generation

Controls complete pipeline stalling.

2. Hazard Stall Support

Handles load-use hazards.

3. Multi-Cycle ALU Support

Handles MUL and DIV execution stalls.

4. Fast Combinational Logic

No clock required.

5. Pipeline Synchronization

Maintains safe execution timing.

Input Signals

Signal Name	Width	Description
hazard_stall	1-bit	Stall request from Hazard Unit
alu_busy	1-bit	Busy signal from ALU multiplier/divider

Output Signals

Signal Name	Width	Description
stall	1-bit	Global pipeline stall signal

Internal Working Principle

The Stall Unit combines multiple stall conditions into one unified signal.

The logic behaves similarly to:
assign stall = hazard_stall || alu_busy;

Hazard Stall Condition

When:
hazard_stall = 1
the pipeline pauses because:

- required operand data is not yet available.

This usually occurs during:
LOAD-use hazard
situations.

ALU Busy Stall Condition

When: alu_busy = 1
the pipeline pauses because:

- multiplication or division is still executing.

This prevents:

- incomplete arithmetic results,
- invalid forwarding,
- and pipeline corruption.

Normal Execution

When:
hazard_stall = 0
alu_busy = 0
then:
stall = 0
The pipeline continues normally.

What Happens During a Stall?

When:
stall = 1
the following blocks freeze:

- Program Counter
- IF/ID Register
- ID/EX Register

This prevents:

- new instruction advancement,
- operand corruption,
- and timing mismatch.

Current execution continues safely until the stall condition clears.

Example Scenario — Hazard Stall

Instruction 1

LOAD R3, [R1]

Instruction 2

ADD R4, R3, R2
ADD requires:
R3
but LOAD has not completed.
Hazard Unit asserts:
hazard_stall = 1
Stall Unit generates:
stall = 1
Pipeline pauses safely.

Example Scenario — Multi-Cycle Stall

Instruction

DIV R1, R7, R2
Divider requires multiple cycles.
During execution:
alu_busy = 1
Pipeline stalls until division completes.

Relationship with Hazard Unit

The Hazard Unit detects:

- operand dependencies.

The Stall Unit converts:
hazard request → full pipeline control

Relationship with ALU

The ALU generates: busy
during:

- multiplication,
- division.

The Stall Unit uses this to synchronize pipeline timing.

Relationship with Pipeline Registers

The Stall Unit directly affects:

- Program Counter
- IF/ID Register
- ID/EX Register

by freezing their updates during stall conditions.

Pipeline Importance

The Stall Unit is extremely important because:

- pipelined processors require precise synchronization,
- multi-cycle operations must complete safely,
- and hazards must not corrupt execution.

Without this block:

- instruction overlap would become unsafe,
- timing would break,
- and processor correctness would fail.

Special Design Advantage

This implementation:

- is simple,
- efficient,
- synthesizable,
- and highly scalable.

It cleanly integrates:

- hazard management,
- multi-cycle execution,
- and pipeline synchronization

into one centralized control mechanism.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Stall Unit
// ========================================
module stall_unit (
    input alu_stall,
    input hazard_stall,
    output stall
);

assign stall = alu_stall | hazard_stall;

endmodule
```

BLOCK 23 — EX/MEM PIPELINE REGISTER

What is this Block?

The EX/MEM Pipeline Register is a sequential storage block placed between the Execute (EX) stage and the Memory Access (MEM) stage of the processor pipeline.
It temporarily stores:

- ALU results,
- memory control signals,
- register information,
- stack operation signals,
- and processor flags

before forwarding them to the Memory stage.
This block acts as the synchronization boundary between:
Execute → Memory
pipeline stages.

Why is this Block Used?

In a pipelined processor:

- different stages operate simultaneously,
- data must remain stable between stages,
- and execution timing must remain synchronized.

Without pipeline registers:

- ALU outputs would change unpredictably,
- memory accesses could become invalid,
- and control signals could mismatch.

The EX/MEM register ensures:

- stable execution results,
- synchronized memory operations,
- and safe pipeline progression.

Role in the Processor

The EX/MEM Pipeline Register performs the following tasks:

- Stores ALU execution result.
- Stores memory access control signals.
- Stores destination register address.
- Stores stack operation controls.
- Transfers stable data to MEM stage.

This block forms the transition point between: execution logic and memory subsystem inside the processor.

Key Features

1. Pipeline Synchronization
Separates EX and MEM stages.

2. Stable Result Transfer
Ensures memory stage receives valid ALU outputs.

3. Memory Control Propagation
Transfers memory read/write signals safely.

4. Stack Operation Support
Supports PUSH, POP, CLEAR, and PEAK instructions.

5. Sequential Clocked Storage
Updates only on a positive clock edge.

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock
reset	1-bit	Clears pipeline register
alu_result_in	16-bit	Result from ALU
rs2_data_in	16-bit	Store data operand
rd_in	3-bit	Destination register
reg_write_in	1-bit	Register write enable
mem_read_in	1-bit	Memory read enable
mem_write_in	1-bit	Memory write enable
mem_to_reg_in	1-bit	Writeback source select
is_push_in	1-bit	PUSH operation control
is_pop_in	1-bit	POP operation control
is_peak_in	1-bit	PEAK operation control
N_in	1-bit	Negative flag

Z_in	1-bit	Zero flag
C_in	1-bit	Carry flag
V_in	1-bit	Overflow flag

Output Signals

Signal Name	Width	Description
alu_result_out	16-bit	ALU result for MEM stage
rs2_data_out	16-bit	Store data forwarded to memory
rd_out	3-bit	Destination register
reg_write_out	1-bit	Register write enable
mem_read_out	1-bit	Memory read enable
mem_write_out	1-bit	Memory write enable
mem_to_reg_out	1-bit	WB source select
is_push_out	1-bit	PUSH operation control
is_pop_out	1-bit	POP operation control
is_peak_out	1-bit	PEAK operation control
N_out	1-bit	Stored Negative flag
Z_out	1-bit	Stored Zero flag
C_out	1-bit	Stored Carry flag
V_out	1-bit	Stored Overflow flag

Internal Working Principle

The EX/MEM register captures Execute-stage outputs on every positive clock edge.

Case 1 — Reset Condition

When: reset = 1
all outputs become zero.
This safely initializes the Memory stage.

Case 2 — Normal Execution

During normal operation:

- ALU results are latched,
- control signals are stored,
- memory operation information is preserved,
- then forwarded to the MEM stage.

Data Stored Inside This Register

This register stores three major categories of information.

1. Execution Results

Includes:

- ALU output
- store operand data

These are required for:

- LOAD
- STORE
- stack operations
- writeback

2. Control Signals

Includes:

- mem_read
- mem_write
- reg_write
- mem_to_reg

These signals control Memory and Writeback stages.

3. Processor Status Information

Includes:

- NZCV flags
- stack operation controls

This preserves execution state consistency across stages.

Relationship with ALU

The ALU produces:
alu_result_in
which is stored inside the EX/MEM register before reaching Data Memory.

Relationship with Data Memory

The Memory stage uses outputs from this register for:

- memory addressing,
- store data,
- stack access,
- and memory control.

Relationship with Stack Operations

For:

- PUSH
- POP
- PEAK

the EX/MEM register safely transfers:

- stack control signals,
- memory addresses,
- and data values

into the Memory stage.

Pipeline Importance

The EX/MEM register is extremely important because:

- Memory stage depends entirely on stable execution outputs,
- memory timing must remain synchronized,
- and pipeline correctness requires isolated stage operation.

Without this block:

- memory accesses could corrupt,
- control signals could mismatch,
- and pipeline timing would fail.

Timing Importance

This register helps:

- break long combinational paths,
- improve timing closure,
- and stabilize pipeline operation.

This is especially important for:

- ASIC synthesis,
- placement,
- and routing.

Special Design Advantage

This implementation:

- cleanly separates Execute and Memory stages,
- supports stack operations,
- preserves processor flags,
- and integrates efficiently into the pipelined datapath.

The architecture is fully synthesizable and timing-friendly.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : EX/MEM Pipeline Register
// ========================================
module ex_mem_reg (
    input clk,
    input reset,
    input stall,

    input [15:0] result_in,
    input [2:0] rd_in,
    input [15:0] rs2_in,
    input reg_write_in,
    input mem_read_in,
    input mem_write_in,
    input mem_to_reg_in,
    input branch_taken_in,
    input [7:0] branch_addr_in,
    input halt_in,

    input is_push_in,
    input is_pop_in,
    input is_peak_in,

    output reg halt_out,
    output reg [15:0] result_out,
    output reg [2:0] rd_out,
    output reg [15:0] rs2_out,
    output reg reg_write_out,
    output reg mem_read_out,
    output reg mem_write_out,
    output reg mem_to_reg_out,
    output reg branch_taken_out,
    output reg [7:0] branch_addr_out,

    output reg is_push_out,
    output reg is_pop_out,
    output reg is_peak_out
);
```

```
always @(posedge clk or posedge reset) begin
    if (reset) begin
        result_out <= 0;
        rd_out <= 0;
        rs2_out <= 0;
        halt_out <= 0;
        reg_write_out <= 0;
        mem_read_out <= 0;
        mem_write_out <= 0;
        mem_to_reg_out <= 0;
        branch_taken_out <= 0;
        branch_addr_out <= 0;

        is_push_out <= 0;
        is_pop_out  <= 0;
        is_peak_out <= 0;
    end
    else if (stall) begin
        // HOLD → do nothing
    end
    else begin
        result_out <= result_in;
        rd_out <= rd_in;
        rs2_out <= rs2_in;
        halt_out <= halt_in;
        reg_write_out <= reg_write_in;
        mem_read_out <= mem_read_in;
        mem_write_out <= mem_write_in;
        mem_to_reg_out <= mem_to_reg_in;
        branch_taken_out <= branch_taken_in;
        branch_addr_out <= branch_addr_in;

        is_push_out <= is_push_in;
        is_pop_out  <= is_pop_in;
        is_peak_out <= is_peak_in;
    end
end

endmodule
```

STAGE 4 — MEMORY ACCESS STAGE (MEM)

BLOCK 24 — DATA MEMORY (DMEM)

What is this Block?

The Data Memory (DMEM) is the memory block used for storing and retrieving runtime data during processor execution. Unlike Instruction Memory, which stores program instructions, Data Memory stores:

- variables,
- temporary computation results,
- stack data,
- and memory operands.

This block supports:

- LOAD operations,
- STORE operations,
- PUSH,
- POP,
- CLEAR,
- and PEAK instructions.

It acts as the main runtime storage system of the processor.

Why is this Block Used?

The processor requires a memory system to store data dynamically during execution.
Registers alone are not sufficient because:

- they are limited in number,
- temporary data can exceed register capacity,
- and stack operations require memory support.

Data Memory allows the processor to:

- store large amounts of data,
- retrieve operands later,
- and support memory-based instructions.

Without this block:

- LOAD and STORE instructions would fail,
- stack operations would not work,
- and the processor would lose practical functionality.

Role in the Processor

The Data Memory performs the following tasks:

- Stores runtime processor data.
- Handles memory read operations.
- Handles memory write operations.
- Supports stack-based memory access.

- Interfaces with the Memory stage of the pipeline.

This block forms the primary data storage subsystem of the processor.

Key Features

1. 16-bit Data Storage

Supports full processor datapath width.

2. Memory Read Support

Supports LOAD and POP operations.

3. Memory Write Support

Supports STORE and PUSH operations.

4. Stack-Compatible Access

Works directly with Stack Pointer logic.

5. Synchronous Write Operation

Ensures stable memory behavior.

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock
mem_read	1-bit	Enables memory read
mem_write	1-bit	Enables memory write
addr	8-bit	Memory address
write_data	16-bit	Data to write into memory

Output Signals

Signal Name	Width	Description
read_data	16-bit	Data read from memory

Internal Working Principle

The Data Memory supports:

- synchronous writing,
- and combinational reading.

Memory Write Operation

When:
mem_write = 1
the processor stores:
write_data
into:
memory[addr]
This operation occurs on:
positive clock edge

Memory Read Operation

When:
mem_read = 1
the memory outputs:
memory[addr]
through:
read_data
This allows:

- LOAD instructions,
- POP operations,
- and stack access.

LOAD Instruction Example

Instruction:
LOAD R3, [R1]
Operation:

- The address comes from the ALU result.
- Data Memory reads stored value.
- Data forwarded to Writeback stage.

STORE Instruction Example

Instruction:
STORE R7, [R1]
Operation:

- The address comes from the ALU result.
- Register data written into memory.

Stack Operation Support

The Data Memory also supports:

- PUSH
- POP

- PEAK
- CLEAR

using addresses generated from:
Stack Pointer (SP)
This allows stack storage inside normal memory space.

Relationship with EX/MEM Register

The EX/MEM register provides:

- memory address,
- write data,
- and memory control signals

to Data Memory.

Relationship with MEM/WB Register

After memory access:

- read data is stored into MEM/WB register,
- then forwarded to the Writeback stage.

Pipeline Importance

The Data Memory is critical because:

- memory instructions are fundamental to processor operation,
- stack functionality depends on it,
- and runtime data storage is essential for practical execution.

Incorrect memory behavior can corrupt:

- variables,
- stack data,
- arithmetic results,
- and processor execution flow.

Timing Importance

Memory blocks are often timing-critical in processor design.
The pipeline architecture helps isolate:

- memory delay,
- execution delay,
- and writeback timing.

This improves:

- synthesis quality,
- timing closure,
- and ASIC implementation reliability.

Special Design Advantage

This implementation:

- cleanly integrates with pipeline,
- supports stack operations efficiently,
- uses lightweight memory architecture,
- and remains highly synthesizable.

The design demonstrates realistic memory subsystem integration for pipelined processor architectures.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Data Memory (DMEM)
// ========================================
module data_memory (
    input clk,
    input reset,
    input mem_read,
    input mem_write,
    input [7:0] addr,
    input [15:0] write_data,
    output reg [15:0] read_data
);

reg [15:0] memory [0:255];
integer i;

// WRITE + RESET
always @(posedge clk or posedge reset) begin
    if (reset) begin
        for (i = 0; i < 256; i = i + 1)
            memory[i] <= 16'd0;
    end
    else if (mem_write) begin
        memory[addr] <= write_data;
    end
end

// READ (FIXED)
always @(posedge clk or posedge reset) begin
    if (reset)
        read_data <= 16'd0;
    else if (mem_read)
        read_data <= memory[addr];
    else
        read_data <= 16'd0;
end

endmodule
```

BLOCK 25 — MEM/WB PIPELINE REGISTER

What is this Block?

The MEM/WB Pipeline Register is a sequential storage block placed between the Memory Access (MEM) stage and the Writeback (WB) stage of the processor pipeline.
It temporarily stores:

- memory read data,
- ALU results,
- destination register information,
- and writeback control signals

before forwarding them to the final Writeback stage.
This block acts as the synchronization boundary between:
Memory → Writeback
stages of the processor pipeline.

Why is this Block Used?

In a pipelined processor:

- each stage operates simultaneously,
- signals must remain stable,
- and timing synchronization is critical.

Without pipeline registers:

- memory outputs could change unpredictably,
- writeback data could become corrupted,
- and register updates could fail.

The MEM/WB register ensures:

- stable transfer of execution results,
- synchronized writeback timing,
- and correct register updates.

Role in the Processor

The MEM/WB Pipeline Register performs the following tasks:

- Stores memory read data.
- Stores ALU execution result.
- Stores destination register address.
- Stores writeback control signals.
- Transfers stable data to WB stage.

This block forms the final synchronization stage before Register File update.

Key Features

1. Pipeline Synchronization
Separates MEM and WB stages.

2. Stable Writeback Data Transfer
Ensures Register File receives stable data.

3. Memory and ALU Result Support
Stores both execution and memory outputs.

4. Writeback Control Preservation
Maintains correct WB behavior.

5. Sequential Clocked Operation
Updates only on a positive clock edge.

Input Signals

Signal Name	Width	Description
clk	1-bit	System clock
reset	1-bit	Clears pipeline register
mem_data_in	16-bit	Data read from Data Memory
alu_result_in	16-bit	ALU execution result
rd_in	3-bit	Destination register address
reg_write_in	1-bit	Register write enable
mem_to_reg_in	1-bit	Writeback source select

Output Signals

Signal Name	Width	Description
mem_data_out	16-bit	Memory data for WB stage
alu_result_out	16-bit	ALU result for WB stage
rd_out	3-bit	Destination register address

reg_write_out	1-bit	Register write enable
mem_to_reg_out	1-bit	Writeback source select

Internal Working Principle

The MEM/WB register captures Memory-stage outputs on every positive clock edge.

Case 1 — Reset Condition

When: reset = 1
all outputs become zero.
This safely initializes the Writeback stage.

Case 2 — Normal Pipeline Operation

During normal execution:

- memory read data is latched,
- ALU result is stored,
- destination register information is preserved,
- writeback control signals are transferred.

These outputs are then forwarded into the WB stage.

Data Stored Inside This Register

The MEM/WB register stores two major categories of information.

1. Execution Results

Includes:

- ALU result
- memory read data

These represent the final processor outputs before register update.

2. Writeback Control Information

Includes:

- reg_write
- mem_to_reg
- destination register address

These determine:

- whether Register File updates,
- and which data source should be written.

Relationship with Data Memory

The Data Memory provides: mem_data_in
for:

- LOAD
- POP
- PEAK

operations.
The MEM/WB register safely transfers this data into the Writeback stage.

Relationship with ALU

For arithmetic and logic instructions:
alu_result_in
is forwarded directly toward Register File writeback.

Relationship with WB MUX

The Writeback MUX uses:

- memory data,
- and ALU result

stored inside the MEM/WB register to select the final writeback value.

Pipeline Importance

The MEM/WB register is critical because:

- Writeback stage requires stable final results,
- register updates must remain synchronized,
- and incorrect WB timing can corrupt processor state.

Without this block:

- Register File writes could become unstable,
- memory results could mismatch,
- and pipeline reliability would fail.

Timing Importance

This register helps:

- isolate memory delay,
- stabilize writeback timing,
- and improve overall pipeline synchronization.

This is especially important during:

- synthesis,
- timing closure,
- and ASIC backend implementation.

Example Operation

LOAD Instruction

Instruction:
LOAD R3, [R1]
Flow:

- Memory data enters the MEM/WB register.
- WB stage later writes data into:

R3

ADD Instruction

Instruction:
ADD R5, R1, R2
Flow:

- ALU result enters MEM/WB register.
- Result forwarded into Register File.

Special Design Advantage

This implementation:

- cleanly separates Memory and Writeback stages,
- preserves pipeline synchronization,
- improves timing stability,
- and remains fully synthesizable.

The architecture follows standard pipelined processor design methodology used in practical CPU implementations.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : MEM/WB Pipeline Register
// ========================================
module mem_wb_reg (
    input clk,
    input reset,

    input [15:0] mem_data_in,
    input [15:0] alu_result_in,
    input [2:0] rd_in,
    input reg_write_in,
    input mem_to_reg_in,
    input halt_in,

    output reg halt_out,
    output reg [15:0] mem_data_out,
    output reg [15:0] alu_result_out,
    output reg [2:0] rd_out,
```

```
    output reg reg_write_out,
    output reg mem_to_reg_out
);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        mem_data_out <= 0;
        alu_result_out <= 0;
        rd_out <= 0;
        halt_out <= 0;
        reg_write_out <= 0;
        mem_to_reg_out <= 0;
    end
    else begin
        mem_data_out <= mem_data_in;
        alu_result_out <= alu_result_in;
        rd_out <= rd_in;
        halt_out <= halt_in;
        reg_write_out <= reg_write_in;
        mem_to_reg_out <= mem_to_reg_in;
    end
end

endmodule
```

BLOCK 26 — WRITEBACK MUX (WB MUX)

What is this Block?

The Writeback MUX (WB MUX) is a multiplexer used in the Writeback stage to select the final data that will be written back into the Register File.
The processor can generate data from different sources such as:

- ALU computation,
- Data Memory,
- stack operations,
- and immediate operations.

Only one final value can be written into a register at a time.
The WB MUX selects the correct final result for register update.

Why is this Block Used?

Different instructions produce results from different hardware blocks.
Example:

ADD Instruction

Result comes from: ALU

LOAD Instruction

Result comes from:
Data Memory
The Register File cannot directly connect to multiple sources simultaneously.
The WB MUX solves this problem by:

- selecting one valid data source,
- isolating inactive sources,
- and forwarding the correct result into the Register File.

Without this block:

- multiple outputs would conflict,
- incorrect data could enter registers,
- and processor state would become corrupted.

Role in the Processor

The Writeback MUX performs the following tasks:

- Selects final register writeback data.
- Chooses between ALU result and memory data.
- Supports LOAD and arithmetic instructions.
- Interfaces directly with Register File.

This block forms the final data-selection stage of the processor pipeline.

Key Features

1. Multi-Source Data Selection

Supports:

- ALU result
- memory read data

2. Instruction-Controlled Operation

Selection depends on instruction type.

3. Fully Combinational Design

No clock required.

4. Pipeline-Compatible

Works directly in the Writeback stage.

5. Lightweight Hardware

Very low logic complexity.

Input Signals

Signal Name	Width	Description
alu_result	16-bit	Result from ALU
mem_data	16-bit	Data from Data Memory
mem_to_reg	1-bit	Writeback source select

Output Signals

Signal Name	Width	Description
writeback_data	16-bit	Final data written into Register File

Internal Working Principle

The WB MUX selects output data depending on:
mem_to_reg
control signal.

Case 1 — ALU Writeback

When:
mem_to_reg = 0

the multiplexer selects:
alu_result
This is used for:

- ADD
- SUB
- MUL
- DIV
- logic operations
- shift operations

and most ALU instructions.

Case 2 — Memory Writeback

When:
mem_to_reg = 1
the multiplexer selects:
mem_data
This is used for:

- LOAD
- POP
- PEAK

operations.

Example Operations

Arithmetic Instruction

Instruction:
ADD R5, R1, R2
Writeback source:
ALU result
Selection:
mem_to_reg = 0

Memory Instruction

Instruction:
LOAD R3, [R1]
Writeback source:
Data Memory
Selection:
mem_to_reg = 1

Relationship with MEM/WB Register

The MEM/WB register provides:

- memory data,

- ALU result,
- and mem_to_reg control signal

to the WB MUX.
The WB MUX then selects the final value for register update.

Relationship with Register File

The output:
writeback_data
is directly connected to:
Register File write_data
input.
This updates destination register:
rd
during the Writeback stage.

Pipeline Importance

The Writeback stage is the final stage of instruction execution.
The WB MUX is critical because:

- it determines final processor state update,
- incorrect selection corrupts register values,
- and pipeline correctness depends on accurate writeback.

Without this block:

- registers could receive invalid data,
- instruction results would become incorrect,
- and processor execution would fail.

Timing Importance

The WB MUX helps:

- isolate data sources,
- stabilize register write timing,
- and simplify writeback datapath.

This improves:

- synthesis quality,
- timing closure,
- and hardware organization.

Special Design Advantage

This implementation:

- is simple,
- synthesizable,

- area-efficient,
- and highly reliable.

The architecture follows standard processor writeback techniques used in practical pipelined CPUs.

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : Writeback MUX (WB MUX)
// ========================================
module writeback_mux (
    input [15:0] alu_result,
    input [15:0] mem_data,
    input mem_to_reg,
    output [15:0] write_data
);

assign write_data = (mem_to_reg) ? mem_data : alu_result;

endmodule
```

TOP LEVEL INTEGRATION

BLOCK 27 — CPU TOP MODULE

What is this Block?

The CPU Top Module is the highest-level integration block of the entire processor.
It connects all major hardware blocks together into one fully functional pipelined CPU system.
This module acts as the:
complete processor architecture
that integrates:

- datapath,
- control path,
- memories,
- pipeline registers,
- hazard handling,
- forwarding logic,
- ALU subsystem,
- and writeback system.

All processor execution begins and ends through this top-level module.

Why is this Block Used?

Individual hardware blocks cannot function independently as a processor.
For example:

- ALU alone cannot fetch instructions.
- Memory alone cannot execute operations.
- Register File alone cannot control execution.

All hardware modules must work together in a synchronized architecture.
The CPU Top Module is used to:

- connect all pipeline stages,
- manage signal flow,
- coordinate execution,
- and implement complete processor functionality.

Without this module:

- the processor would exist only as disconnected hardware blocks,
- and complete execution would not be possible.

Role in the Processor

The CPU Top Module performs the following tasks:

- Integrates entire processor datapath.
- Connects all pipeline stages.
- Controls instruction execution flow.
- Manages pipeline synchronization.

- Coordinates hazard handling.
- Controls memory and writeback flow.

This module represents the complete processor system.

Integrated Pipeline Stages

The processor implements a:
5-Stage Pipeline Architecture
consisting of:

Stage	Description
IF	Instruction Fetch
ID	Instruction Decode
EX	Execute
MEM	Memory Access
WB	Writeback

Stage 1 — Instruction Fetch (IF)

This stage:

- fetches instructions from Instruction Memory,
- updates Program Counter,
- handles branch redirection.

Integrated Blocks

- Program Counter (PC)
- Instruction Memory
- IF/ID Pipeline Register
- Flush Unit

Stage 2 — Instruction Decode (ID)

This stage:

- decodes instruction fields,
- reads register operands,
- generates control signals,
- detects hazards.

Integrated Blocks

- Instruction Decoder
- Register File

- Immediate Generator
- Control Unit
- Hazard Detection Unit
- ID/EX Pipeline Register

Stage 3 — Execute (EX)

This stage:

- performs arithmetic and logic operations,
- handles forwarding,
- evaluates branches,
- executes MUL and DIV operations.

Integrated Blocks

- Forwarding Unit
- Forward MUX A
- Forward MUX B
- Operand MUX
- Brent-Kung Adder
- Shift-Add Multiplier
- Restoring Divider
- Pipelined ALU
- Flag Register
- Branch Unit
- Stall Unit
- EX/MEM Pipeline Register

Stage 4 — Memory Access (MEM)

This stage:

- accesses Data Memory,
- performs LOAD/STORE operations,
- handles stack memory operations.

Integrated Blocks

- Stack Pointer (SP)
- Data Memory
- MEM/WB Pipeline Register

Stage 5 — Writeback (WB)

This stage:

- selects final result,
- updates Register File.

Integrated Blocks

- Writeback MUX

Key Features of Complete Processor

1. 5-Stage Pipelined Architecture

Improves instruction throughput using overlapping execution.

2. Hazard Handling Support

Supports:

- forwarding,
- stalling,
- and flushing.

3. Multi-Cycle MUL/DIV Execution

Supports advanced arithmetic operations safely.

4. NZCV Flag System

Supports conditional branching and comparison operations.

5. Stack Operation Support

Supports:

- PUSH
- POP
- PEAK

instructions.

6. Custom 16-bit ISA

Implements custom instruction architecture.

7. RTL-to-GDSII Compatible Design

Fully synthesizable processor architecture.

Major Internal Connections

The CPU Top Module connects:

Fetch Path

PC → Instruction Memory → IF/ID Register

Decode Path

Instruction Decoder → Control Unit → Register File

Execute Path
Forwarding → Operand Selection → ALU

Memory Path
ALU Result → Data Memory

Writeback Path
WB MUX → Register File

Hazard Handling Integration
The processor integrates three major hazard-handling mechanisms.

1. Forwarding
Handled using:
- Forwarding Unit
- Forward MUXes

Prevents unnecessary stalls.

2. Hazard Detection
Handled using:
- Hazard Detection Unit

Detects load-use dependencies.

3. Pipeline Flushing
Handled using:
- Branch Unit
- Flush Unit

Removes wrong-path instructions after branch execution.

Multi-Cycle Execution Integration
The processor supports:
- MUL
- DIV

using:
- Shift-Add Multiplier
- Restoring Divider

During execution:
busy = 1
The pipeline temporarily stalls until operation completes.
This demonstrates advanced pipeline synchronization capability.

Instruction Flow Through Pipeline

Step 1 — Fetch

Instruction fetched from Instruction Memory.

Step 2 — Decode

Instruction fields extracted and control generated.

Step 3 — Execute

ALU performs computation.

Step 4 — Memory Access

LOAD/STORE operations performed.

Step 5 — Writeback

Final result written into Register File.

Processor Importance

The CPU Top Module represents:

- the complete processor architecture,
- the full datapath integration,
- and the final hardware implementation.

It combines all individual modules into one functioning processor system capable of:

- arithmetic computation,
- logic execution,
- memory operations,
- stack handling,
- hazard handling,
- and conditional branching.

Design Strength of This Processor

This processor demonstrates:

- realistic pipelined architecture,
- advanced hazard handling,
- multi-cycle arithmetic support,
- ASIC-oriented RTL design,
- modular datapath organization,
- and synthesizable hardware implementation.

The architecture is significantly more advanced than simple educational single-cycle CPUs.

Final Processor Capability Summary

Capability	Supported
5-Stage Pipeline	Yes
Hazard Detection	Yes
Forwarding	Yes
Pipeline Flush	Yes
MUL/DIV	Yes
Stack Operations	Yes
Branching	Yes
NZCV Flags	Yes
RTL Synthesis	Yes
GDSII Implementation	Yes

Verilog RTL Code

```
// ========================================
// Verilog RTL Code : CPU Top Module
// ========================================
module cpu_top (
    input clk,
    input reset,

    output [15:0] debug_result,
    output [2:0]  debug_rd,
    output        debug_reg_write,
    output [15:0] debug_mem_data,
    output [3:0]  flags_out_debug,
    output [15:0] debug_alu_result_ex,
    output [15:0] debug_rs1_ex,
    output [15:0] debug_rs2_ex,
    output [7:0]  debug_sp
);
//=============================================================//
//                  STAGE 1 : IF                               //
//=============================================================//

wire [7:0] pc;
wire [15:0] instr;
```

```
// Control signals from later stages
wire stall;
wire halt;
wire branch_taken;
wire [7:0] branch_addr;

wire [7:0] pc_ex;
wire [15:0] rs1_ex, rs2_ex, imm_ex;
wire [2:0] rs1_addr_ex, rs2_addr_ex;
wire [2:0] opcode_ex;
wire [2:0] rd_ex;

wire [2:0] alu_op_ex;
wire alu_src_ex, reg_write_ex, mem_read_ex, mem_write_ex, mem_to_reg_ex;
wire jump_ex, flag_write_ex;
wire halt_ex, halt_mem, halt_wb;
wire is_push, is_pop, is_peak;

wire [15:0] result_mem, rs2_mem;
wire [2:0] rd_mem;
wire reg_write_mem, mem_read_mem, mem_write_mem, mem_to_reg_mem;
wire is_push_ex, is_pop_ex, is_peak_ex;

// Program Counter
program_counter PC (
    .clk(clk),
    .reset(reset),
    .stall(stall),
    .halt(halt_wb),
    .branch_taken(branch_taken),
    .branch_addr(branch_addr),
    .pc(pc)
);

// Instruction Memory
instruction_memory IMEM (
    .addr(pc),
    .instr(instr)
);

// Flush logic
wire flush;
flush_unit FLUSH (
    .branch_taken(branch_taken),
    .flush(flush)
);

// IF/ID Pipeline Register
wire [7:0] pc_id;
wire [15:0] instr_id;
```

```
if_id_reg IF_ID (
    .clk(clk),
    .reset(reset),
    .stall(stall),
    .flush(flush),
    .pc_in(pc),
    .instr_in(instr),
    .pc_out(pc_id),
    .instr_out(instr_id)
);

//=============================================================//
//                  STAGE 2 : ID                               //
//=============================================================//

// Decode instruction
wire [1:0] group;
wire [2:0] opcode, rd, rs1, rs2;
wire [4:0] imm;

instruction_decoder DEC (
    .instr(instr_id),
    .group(group),
    .opcode(opcode),
    .rd(rd),
    .rs1(rs1),
    .rs2(rs2),
    .imm(imm)
);

// Control signals
wire alu_src, reg_write, mem_read, mem_write, mem_to_reg;
wire jump, flag_write;
wire [2:0] alu_op;

control_unit CTRL (
    .group(group),
    .opcode(opcode),

    .is_push(is_push),
    .is_pop(is_pop),
    .is_peak(is_peak),

    .alu_src(alu_src),
    .reg_write(reg_write),
    .mem_read(mem_read),
    .mem_write(mem_write),
    .mem_to_reg(mem_to_reg),
    .jump(jump),
    .halt(halt),
```

```
    .alu_op(alu_op),
    .flag_write(flag_write)
);

// Register File
wire [15:0] rs1_data, rs2_data;
wire [15:0] writeback_data;
wire reg_write_wb;
wire [2:0] rd_wb;

register_file RF (
    .clk(clk),
    .reset(reset),
    .reg_write(reg_write_wb),
    .rs1(rs1),
    .rs2(rs2),
    .rd(rd_wb),
    .write_data(writeback_data),
    .read_data1(rs1_data),
    .read_data2(rs2_data)
);
// Immediate Generator
wire [15:0] imm_ext;

immediate_generator IMM (
    .imm_in(imm),
    .imm_out(imm_ext)
);

// Hazard Detection
wire hazard_stall;

hazard_unit HAZARD (
    .rs1_id(rs1),
    .rs2_id(rs2),
    .rd_ex(rd_ex),
    .mem_read_ex(mem_read_ex),
    .hazard_stall(hazard_stall)
);

//=============================================================//
//                  ID/EX REGISTER                             //
//=============================================================//
wire [1:0] group_ex;

id_ex_reg ID_EX (
    .clk(clk),
    .reset(reset),
    .stall(stall),
    .flush(flush),
```

```
    .pc_in(pc_id),
    .rs1_in(rs1_data),
    .rs2_in(rs2_data),
    .imm_in(imm_ext),
    .rd_in(rd),
    .group_in(group),
    .halt_in(halt),

    .halt_out(halt_ex),
    .group_out(group_ex),
    .alu_op_in(alu_op),
    .alu_src_in(alu_src),
    .reg_write_in(reg_write),
    .mem_read_in(mem_read),
    .mem_write_in(mem_write),
    .mem_to_reg_in(mem_to_reg),
    .jump_in(jump),
    .flag_write_in(flag_write),
    .rs1_addr_in(rs1),
    .rs2_addr_in(rs2),
    .opcode_in(opcode),

    .opcode_out(opcode_ex),
    .rs1_addr_out(rs1_addr_ex),
    .rs2_addr_out(rs2_addr_ex),
    .pc_out(pc_ex),
    .rs1_out(rs1_ex),
    .rs2_out(rs2_ex),
    .imm_out(imm_ex),
    .rd_out(rd_ex),

    .alu_op_out(alu_op_ex),
    .alu_src_out(alu_src_ex),
    .reg_write_out(reg_write_ex),
    .mem_read_out(mem_read_ex),
    .mem_write_out(mem_write_ex),
    .mem_to_reg_out(mem_to_reg_ex),
    .jump_out(jump_ex),
    .flag_write_out(flag_write_ex),
    .is_push_in(is_push),
    .is_pop_in(is_pop),
    .is_peak_in(is_peak),

    .is_push_out(is_push_ex),
    .is_pop_out(is_pop_ex),
    .is_peak_out(is_peak_ex)
);

//=============================================================//
//                  STAGE 3 : EX                               //
//=============================================================//
```

```
// Forwarding
wire [1:0] forwardA, forwardB;

forwarding_unit FWD (
    .rs1_ex(rs1_addr_ex),
    .rs2_ex(rs2_addr_ex),
    .rd_mem(rd_mem),
    .rd_wb(rd_wb),
    .reg_write_mem(reg_write_mem),
    .reg_write_wb(reg_write_wb),
    .forwardA(forwardA),
    .forwardB(forwardB)
);

// Forward mux
wire [15:0] opA, opB_raw;

forward_mux FMUX_A (
    .reg_data(rs1_ex),
    .ex_mem_data(result_mem),
    .mem_wb_data(writeback_data),
    .forward_sel(forwardA),
    .out(opA)
);

forward_mux FMUX_B (
    .reg_data(rs2_ex),
    .ex_mem_data(result_mem),
    .mem_wb_data(writeback_data),
    .forward_sel(forwardB),
    .out(opB_raw)
);

// Operand mux (imm vs reg)
wire [15:0] opB;

operand_mux OPMUX (
    .reg_data(opB_raw),
    .imm_data(imm_ex),
    .alu_src(alu_src_ex),
    .operand_out(opB)
);

// ALU
wire [15:0] alu_result;
wire [3:0] flags;
wire alu_stall;

// Detect special instructions
wire is_move  = (group_ex == 2'b10) && (alu_op_ex == 3'b010);
```

```
wire is_loadi = (group_ex == 2'b10) && (alu_op_ex == 3'b011);
wire is_clear = (group_ex == 2'b10) && (alu_op_ex == 3'b111);

// LOADI fix
wire [15:0] opA_final = is_loadi ? 16'd0 : opA;

pipelined_alu ALU (
    .clk(clk),
    .rst(reset),
    .A(opA_final),
    .B(opB),
    .opcode({group_ex, alu_op_ex[2:0]}),
    .flags(flags),
    .result(alu_result),
    .stall(alu_stall)
);

wire [15:0] final_ex_result =
    is_clear ? 16'd0 :
    is_move  ? opA :
    alu_result;

// Flag register
wire [3:0] flags_out;

// ==================
// FLAG PIPELINE STAGE (FIXED)
// ==================
reg [3:0] flags_stage;
reg       flag_write_stage;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        flags_stage <= 4'b0;
        flag_write_stage <= 1'b0;
    end
    else if (!alu_stall) begin   // FIXED (was !stall)
        flags_stage <= flags;
        flag_write_stage <= flag_write_ex & ~alu_stall;
    end
end

// ==================
// FINAL FLAG REGISTER
// ==================
flag_register FLAGS (
    .clk(clk),
    .reset(reset),
    .flag_write(flag_write_stage),
    .flags_in(flags_stage),
    .flags_out(flags_out)
```

```
);
assign debug_result    = writeback_data;
assign debug_rd        = rd_wb;
assign debug_reg_write = reg_write_wb;
assign flags_out_debug = flags_out;

// ==================
// BRANCH PIPELINE STAGE (NEW)
// ==================
reg [4:0] opcode_branch_d;
reg [7:0] pc_branch_d;
reg [7:0] imm_branch_d;
reg       jump_branch_d;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        opcode_branch_d <= 5'b0;
        pc_branch_d <= 8'b0;
        imm_branch_d <= 8'b0;
        jump_branch_d <= 1'b0;
    end
    else if (!stall) begin
        opcode_branch_d <= {group_ex, alu_op_ex};
        pc_branch_d <= pc_ex;
        imm_branch_d <= imm_ex[7:0];
        jump_branch_d <= jump_ex;
    end
end

// Branch unit
branch_unit BRANCH (
    .jump(jump_branch_d),
    .opcode(opcode_branch_d),
    .flags_out(flags_out),
    .pc_ex(pc_branch_d),
    .imm_ex(imm_branch_d),
    .branch_taken(branch_taken),
    .branch_addr(branch_addr)
);

// Stall unit (global)
stall_unit STALL (
    .alu_stall(alu_stall),
    .hazard_stall(hazard_stall),
    .stall(stall)
);

//=============================================================//
//                  EX/MEM REGISTER                            //
//=============================================================//
wire is_push_mem, is_pop_mem, is_peak_mem;
```

```
ex_mem_reg EX_MEM (
    .clk(clk),
    .reset(reset),
    .stall(stall),

    .result_in(final_ex_result),
    .rd_in(rd_ex),
    .rs2_in(rs2_ex),
    .reg_write_in(reg_write_ex),
    .mem_read_in(mem_read_ex),
    .mem_write_in(mem_write_ex),
    .mem_to_reg_in(mem_to_reg_ex),
    .branch_taken_in(branch_taken),
    .branch_addr_in(branch_addr),
    .halt_in(halt_ex),

    // NEW
    .is_push_in(is_push_ex),
    .is_pop_in(is_pop_ex),
    .is_peak_in(is_peak_ex),

    .halt_out(halt_mem),
    .result_out(result_mem),
    .rd_out(rd_mem),
    .rs2_out(rs2_mem),
    .reg_write_out(reg_write_mem),
    .mem_read_out(mem_read_mem),
    .mem_write_out(mem_write_mem),
    .mem_to_reg_out(mem_to_reg_mem),

    // NEW
    .is_push_out(is_push_mem),
    .is_pop_out(is_pop_mem),
    .is_peak_out(is_peak_mem)
);

wire [7:0] sp;

stack_pointer SP (
    .clk(clk),
    .reset(reset),
    .push(is_push_mem),
    .pop(is_pop_mem),
    .sp(sp)
);

//============================================================//
//                  STAGE 4 : MEM                             //
//============================================================//
```

```
wire [15:0] mem_data;
reg [7:0] mem_addr_r;

always @(posedge clk or posedge reset) begin
    if (reset)
        mem_addr_r <= 8'd0;
    else
        mem_addr_r <=
            is_push_mem ? sp :
            is_pop_mem  ? (sp - 1) :
            is_peak_mem ? sp :
            result_mem[7:0];
end

data_memory DMEM (
    .clk(clk),
    .reset(reset),
    .mem_read(mem_read_mem),
    .mem_write(mem_write_mem),
    .addr(mem_addr_r),
    .write_data(rs2_mem),
    .read_data(mem_data)
);

// MEM/WB register
wire [15:0] mem_data_wb, alu_result_wb;
wire mem_to_reg_wb;

mem_wb_reg MEM_WB (
    .clk(clk),
    .reset(reset),

    .mem_data_in(mem_data),
    .alu_result_in(result_mem),
    .rd_in(rd_mem),
    .reg_write_in(reg_write_mem),
    .mem_to_reg_in(mem_to_reg_mem),
    .halt_in(halt_mem),

    .halt_out(halt_wb),
    .mem_data_out(mem_data_wb),
    .alu_result_out(alu_result_wb),
    .rd_out(rd_wb),
    .reg_write_out(reg_write_wb),
    .mem_to_reg_out(mem_to_reg_wb)
);

//=============================================================//
//                  STAGE 5 : WB                               //
//=============================================================//
```

```
writeback_mux WB (
    .alu_result(alu_result_wb),
    .mem_data(mem_data_wb),
    .mem_to_reg(mem_to_reg_wb),
    .write_data(writeback_data)
);

reg [15:0] debug_mem_data_r;
reg [15:0] debug_alu_result_ex_r;
reg [15:0] debug_rs1_ex_r;
reg [15:0] debug_rs2_ex_r;
reg [7:0]  debug_sp_r;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        debug_mem_data_r      <= 0;
        debug_alu_result_ex_r <= 0;
        debug_rs1_ex_r        <= 0;
        debug_rs2_ex_r        <= 0;
        debug_sp_r            <= 0;
    end else begin
        debug_mem_data_r      <= mem_data;
        debug_alu_result_ex_r <= alu_result;
        debug_rs1_ex_r        <= rs1_ex;
        debug_rs2_ex_r        <= rs2_ex;
        debug_sp_r            <= sp;
    end
end

assign debug_mem_data      = debug_mem_data_r;
assign debug_alu_result_ex = debug_alu_result_ex_r;
assign debug_rs1_ex        = debug_rs1_ex_r;
assign debug_rs2_ex        = debug_rs2_ex_r;
assign debug_sp            = debug_sp_r;
endmodule
```

PIPELINE ARCHITECTURE

What is Pipeline Architecture?

Pipeline Architecture is a processor design technique where multiple instructions are executed simultaneously in different stages of execution. Instead of completing one instruction fully before starting the next one, the processor divides execution into smaller stages.
Each stage performs a specific task.
This improves:

- instruction throughput,
- processor efficiency,
- and overall performance.

Why is Pipeline Architecture Used?

In a non-pipelined processor:

- one instruction completes fully,
- then the next instruction begins.

This wastes hardware resources because many blocks remain idle during execution.
Pipeline architecture solves this problem by allowing:
multiple instructions to execute in parallel
inside different stages.
This significantly improves processor performance.
Without pipelining:

- execution becomes slower,
- hardware utilization decreases,
- and throughput reduces heavily.

Pipeline Type Used in This Processor

This processor implements a:
5-Stage Pipelined Architecture
The stages are:

Stage	Name	Main Function
IF	Instruction Fetch	Fetch instruction from memory
ID	Instruction Decode	Decode instruction and read operands
EX	Execute	Perform ALU operations
MEM	Memory Access	Access Data Memory
WB	Writeback	Write final result into Register File

Stage 1 — Instruction Fetch (IF)

Purpose

This stage fetches the next instruction from Instruction Memory.

Operations Performed

- Program Counter generates instruction address.
- Instruction Memory outputs instruction.
- The IF/ID register stores fetched instructions.

Main Blocks Used

- Program Counter
- Instruction Memory
- IF/ID Pipeline Register
- Flush Unit

Stage 2 — Instruction Decode (ID)

Purpose

This stage interprets the fetched instruction.

Operations Performed

- Instruction fields extracted.
- Registers read from Register File.
- Immediate values generated.
- Control signals generated.
- Hazards detected.

Main Blocks Used

- Instruction Decoder
- Register File
- Immediate Generator
- Control Unit
- Hazard Detection Unit
- ID/EX Pipeline Register

Stage 3 — Execute (EX)

Purpose

This stage performs actual computation.

Operations Performed

- ALU operations executed.
- Forwarding applied.
- Branch conditions evaluated.
- Multiplication/division executed.
- Flags generated.

Main Blocks Used

- Forwarding Unit
- Forward MUX A
- Forward MUX B
- Operand MUX
- Brent-Kung Adder
- Shift-Add Multiplier
- Restoring Divider
- Pipelined ALU
- Flag Register
- Branch Unit
- Stall Unit
- EX/MEM Pipeline Register

Stage 4 — Memory Access (MEM)

Purpose

This stage performs memory operations.

Operations Performed

- LOAD execution
- STORE execution
- PUSH/POP operations
- Stack memory access

Main Blocks Used

- Stack Pointer
- Data Memory
- MEM/WB Pipeline Register

Stage 5 — Writeback (WB)

Purpose

This stage updates processor registers with final execution result.

Operations Performed

- Select final result.
- Write data into Register File.

Main Blocks Used

- Writeback MUX

Parallel Instruction Execution

Pipeline allows different instructions to execute simultaneously.
Example:

Clock Cycle	IF	ID	EX	MEM	WB
1	I1	-	-	-	-
2	I2	I1	-	-	-
3	I3	I2	I1	-	-
4	I4	I3	I2	I1	-
5	I5	I4	I3	I2	I1

This overlapping execution improves throughput significantly.

Pipeline Registers Used

Pipeline registers separate stages and maintains synchronization.

Pipeline Register	Purpose
IF/ID	Separates IF and ID
ID/EX	Separates ID and EX
EX/MEM	Separates EX and MEM
MEM/WB	Separates MEM and WB

These registers:
- stabilize signals,
- improve timing,
- and enable simultaneous execution.

Pipeline Hazards

Pipelining introduces execution hazards.
This processor handles three major hazard types.

1. Data Hazards

Occurs when one instruction depends on another instruction's result.
Handled using:
- Forwarding Unit
- Hazard Detection Unit

2. Control Hazards

Occurs during branch instructions.
Handled using:
- Branch Unit
- Flush Unit

3. Structural Hazards

Occurs when multiple stages require the same hardware simultaneously.
This processor minimizes structural hazards using:
- separated Instruction Memory and Data Memory,
- dedicated datapath organization.

Multi-Cycle Operation Support

The processor supports:
- MUL
- DIV

as multi-cycle operations.
During execution:
busy = 1
The pipeline temporarily stalls until operation completes safely.
This demonstrates advanced pipeline synchronization capability.

Pipeline Advantages

1. Higher Throughput

Multiple instructions execute simultaneously.

2. Better Hardware Utilization
Hardware blocks remain active more frequently.

3. Improved Performance
The processor completes more instructions per unit time.

4. Modular Design
Each stage performs a specialized function.

5. Better Scalability
Pipeline can be extended for advanced architectures.

Pipeline Challenges
Pipelining also introduces challenges:
- hazard handling,
- synchronization,
- forwarding complexity,
- branch correction,
- and stall management.

This processor implements dedicated hardware solutions for all these challenges.

Importance in Modern Processors
Nearly all modern processors use pipelining because:
- it significantly improves performance,
- increases instruction throughput,
- and enables efficient processor execution.

This project demonstrates a realistic pipelined architecture implementation rather than a basic single-cycle processor.

Special Design Advantage
This pipeline architecture:
- supports hazard handling,
- integrates multi-cycle arithmetic units,
- maintains pipeline synchronization,
- and remains fully synthesizable.

The design demonstrates practical processor engineering concepts used in real CPU architectures.

Pipeline Flow Diagram
IF → ID → EX → MEM → WB

HAZARD HANDLING MECHANISM

What is Hazard Handling?

Hazard Handling is the collection of hardware techniques used to maintain correct instruction execution inside a pipelined processor when instruction conflicts occur.
In pipelined execution:

- multiple instructions execute simultaneously,
- instructions overlap across stages,
- and dependencies can occur between them.

These conflicts are called:
Pipeline Hazards
The processor includes dedicated hardware blocks to detect and resolve these hazards safely.

Why is Hazard Handling Required?

Pipelining improves performance, but overlapping execution introduces dependency problems.
Example:
ADD R5, R1, R2
SUB R6, R5, R3
The second instruction requires:
R5
before the first instruction has completed writeback.
Without hazard handling:

- incorrect operands would be used,
- execution would become invalid,
- and processor results would corrupt.

Hazard handling ensures:

- correct execution order,
- safe pipeline operation,
- and synchronized instruction flow.

Types of Hazards in Pipeline

This processor handles three major hazard categories.

Hazard Type	Cause
Data Hazard	Instruction dependency
Control Hazard	Branch/jump instruction
Structural Hazard	Hardware resource conflict

1. DATA HAZARD

What is Data Hazard?

A Data Hazard occurs when:
one instruction depends on data from another instruction
that has not yet completed execution.

Example

ADD R5, R1, R2
SUB R6, R5, R3
The SUB instruction needs:
R5
before ADD completes writeback.

Types of Data Hazards Handled

This processor mainly handles:
RAW (Read After Write)
hazards.

Hardware Used for Data Hazard Handling

Block	Purpose
Forwarding Unit	Bypasses latest ALU data
Forward MUX A/B	Select forwarded operands
Hazard Detection Unit	Detects load-use hazard
Stall Unit	Freezes pipeline temporarily

Forwarding Mechanism

Forwarding avoids unnecessary stalls by directly bypassing latest results.
Instead of waiting for Register File update:

- The latest ALU result is forwarded directly to the EX stage.

Example

ADD R5, R1, R2
SUB R6, R5, R3
The result of:
ADD
is forwarded directly into:

SUB
ALU input.

Load-Use Hazard

Some hazards cannot be solved using forwarding.
Example:
LOAD R3, [R1]
ADD R4, R3, R2
Memory data is not immediately available.
The processor therefore:

- detects dependency,
- stalls pipeline temporarily,
- then resumes execution safely.

2. CONTROL HAZARD

What is Control Hazard?

A Control Hazard occurs when:

- branch instructions,
- jumps,
- or conditional execution

change processor control flow.

Example

JZ +3
The processor initially does not know whether branch will occur.
Meanwhile:

- next instructions may already enter the pipeline.

If branch becomes taken:

- wrong instructions must be removed.

Hardware Used for Control Hazard Handling

Block	Purpose
Branch Unit	Evaluates branch conditions
Flush Unit	Removes wrong instructions
Program Counter	Redirects execution

Pipeline Flush Mechanism

When:
branch_taken = 1
the processor:

- invalidates incorrect instructions,
- clears wrong pipeline data,
- redirects PC to correct target.

This is called:
Pipeline Flushing

3. STRUCTURAL HAZARD

What is Structural Hazard?

A Structural Hazard occurs when:
multiple pipeline stages require same hardware simultaneously

Structural Hazard Prevention in This Processor

This processor minimizes structural hazards using:

Separate Memories

- Instruction Memory
- Data Memory

This allows:

- instruction fetch,
- and memory access

to occur simultaneously.

Dedicated Functional Units

Separate hardware exists for:

- ALU
- Multiplier
- Divider
- Memory system

This reduces hardware conflicts significantly.

Multi-Cycle Hazard Handling

The processor supports:

- MUL
- DIV

as multi-cycle operations.
During execution:
busy = 1
The Stall Unit:

- freezes pipeline,
- prevents new instruction advancement,
- and resumes execution after completion.

Hazard Resolution Flow

Step 1 — Hazard Detection

Processor checks:

- register dependencies,
- branch conditions,
- multi-cycle busy signals.

Step 2 — Hazard Classification

Processor determines:

- forwarding possible?
- stall required?
- flush required?

Step 3 — Resolution

Processor applies:

- forwarding,
- stalling,
- or flushing.

Example Hazard Resolution

Instruction Sequence

LOAD R3, [R1]
ADD R4, R3, R2

Detection

Hazard Detection Unit detects:
R3 dependency

Action

Stall Unit generates:
stall = 1

Result

Pipeline pauses safely until data becomes available.

Importance of Hazard Handling

Hazard handling is one of the most critical parts of pipelined processor design.
Without hazard handling:

- pipeline execution becomes unreliable,

- instruction results become incorrect,
- and processor behavior becomes unpredictable.

Proper hazard management ensures:

- correctness,
- synchronization,
- and stable execution.

Performance Impact

Good hazard handling significantly improves:

- pipeline efficiency,
- instruction throughput,
- and processor performance.

Forwarding especially reduces:
unnecessary stalls
which improves execution speed.

Advanced Design Aspect

This processor demonstrates advanced pipeline engineering concepts including:

- forwarding,
- load-use hazard detection,
- branch flushing,
- and multi-cycle synchronization.

These are real architectural techniques used in practical modern processors.

Special Design Advantage

This implementation:

- balances performance and correctness,
- minimizes unnecessary stalls,
- integrates cleanly into pipeline,
- and remains fully synthesizable.

The hazard handling system significantly strengthens the processor architecture quality.